From the Perceptron to ChatGPT

How a neural network works, starting from nothing

Cristian Taccioli

MAPS Dept., University of Padova · English version: Dept. of Computer Science and Technology, University of Cambridge, Thomson Avenue, Cambridge, August 2026

How to use these notes. Every time you see the symbol **BY HAND** you must stop, take pencil and paper, and redo the calculation yourself. You will not need a calculator: every number has a single decimal place. If you do not redo the sums, you do not really understand: that is the only rule of this course.
The first twelve pages need nothing beyond lower secondary school mathematics. From the section on derivatives onwards you need to know what a slope is, and the little that is required is explained there.

1. The problem

Imagine you have to decide, every Saturday night, whether to go to the cinema or stay at home. The decision depends on two things only:

- **how many friends are coming**: none, one, two, three, four;
- **how far the cinema is**: the distance from your home, in kilometres.

Saturday after Saturday you have made a decision. Now we want to build a machine that looks at your past decisions, works out your rule on its own, and can decide in your place even in new situations it has never seen.

That machine is called a **perceptron** and it is the elementary building block of every neural network, including the enormous ones that today translate languages, recognise faces or write text. It was invented in 1958 by Frank Rosenblatt, an American psychologist, and his idea was very simple: to copy, in a very crude way, what a neuron of the brain does.

Turning a situation into numbers

A computer does not understand words, it only understands figures. So the first thing to do is to translate everything into numbers. This operation is called **encoding** and it is the first step of any artificial intelligence project.

Variable	Meaning	Values
x_1	number of friends coming	from 0 to 4
x_2	distance from home to the cinema	a number of kilometres, from 1 to 7
y	**the true answer**	$0 =$ I stay in, $1 =$ I go to the cinema

The first two variables are called **inputs** (or *features*): they are the information that enters the machine. The last one, y, is called the **label**, or the **outcome**: it is the right answer, the one you actually gave that Saturday.

Two inputs alone are not a limitation, they are a teaching choice. With two inputs every Saturday is a point on a sheet of graph paper, and we shall be able to draw everything. Later we shall see what happens when the inputs become three, ten or ten thousand.

2. The dataset

Here are the ten Saturdays we shall use to teach the perceptron. This group of data is called the **training set**.

row	x_1 **friends**	x_2 **km**	y	**what happened**
R1	1	1	1	one friend and the cinema round the corner: we went
R2	1	3	0	one friend alone is not worth three kilometres
R3	2	3	1	with two of us, three kilometres are fine
R4	0	3	0	on my own and far away: I stayed in
R5	3	5	1	with three of us we go even five kilometres
R6	2	5	0	with two of us, five kilometres are too many
R7	4	6	1	the whole gang: six kilometres are worth it
R8	3	7	0	with three of us, seven kilometres are too many
R9	0	1	0	on my own I do not go, even if it is close
R10	4	7	1	with four friends even seven kilometres are fine

The dataset is **balanced**: five times we went ($y = 1$) and five times we stayed in ($y = 0$). This matters, because if we had had nine *no* and a single *yes*, the machine could learn the stupid strategy "always answer no" and be wrong only once in ten.

Looking at the table you can already guess the rule: *each extra friend makes me willing to travel a bit further*. But we shall never tell the machine. Its job is to dig it out on its own.

3. What a perceptron is

A perceptron is a little machine that does three things in a row, always the same three.

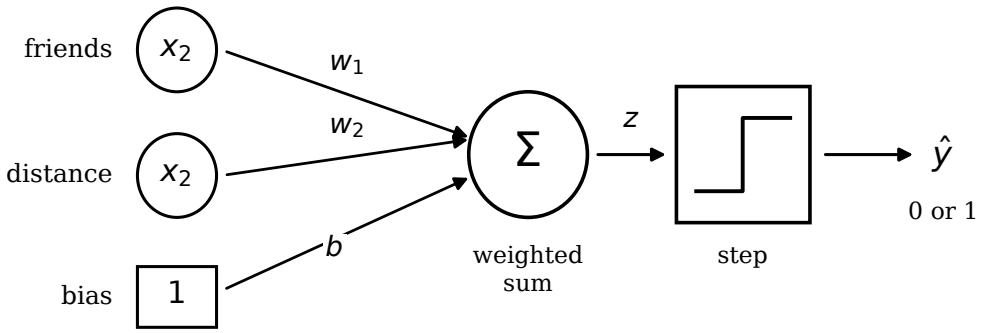


Figure 1: The anatomy of a perceptron. The inputs come in from the left, each is multiplied by its weight, the results are added together along with the bias, and the total goes into the step function that decides 0 or 1.

3.1 First: give a weight to every piece of information

Not all information counts the same. The perceptron attaches to every input x_i a number w_i called a **weight**.

- A **large positive** weight means "this pushes me towards going".
- A **negative** weight means "this holds me back".

- A weight **close to zero** means "this does not interest me".

You would expect the weight of the friends to be positive and the weight of the distance to be negative. We shall see that the machine gets there on its own, but not immediately.

3.2 Second: take the weighted sum and add the bias

Multiply every input by its weight, add everything up, and add a fixed number b called the **bias**:

$$z \;=\; w_1 x_1 \;+\; w_2 x_2 \;+\; b$$

The number z that comes out of this calculation is the **score** of the evening.

3.3 Third: decide with the step function

The score z is any number at all, but the answer must be only yes or no. So we apply the **step function**:

$$\hat{y} \;=\; \begin{cases} 1 & \text{if } z \geq 0 \quad \text{(WE GO)} \\ 0 & \text{if } z < 0 \quad \text{(WE STAY IN)} \end{cases}$$

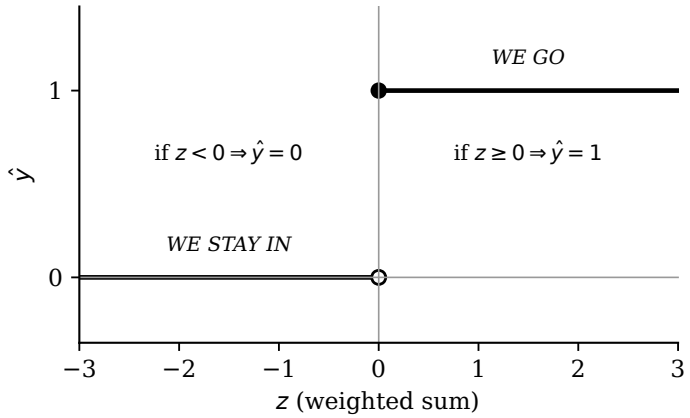


Figure 2: The step function. There is nothing in between: either you are up on the step (answer 1) or you are down (answer 0). The jump happens exactly at $z = 0$.

Where the name comes from

A real neuron receives signals from other neurons through its synapses. Some synapses are strong, others weak: those are the weights. The neuron adds up all the signals it receives and, if the sum passes a certain threshold, it "fires" an electrical impulse; if it does not, it stays quiet. It never fires halfway. This all-or-nothing behaviour is exactly the step function. The bias is the threshold, moved to the other side of the equals sign.

The symbol $\hat{y}$

We write y for the *true* answer, the one written in the dataset, and $\hat{y}$ (read "y hat") for the answer *guessed by the machine*. When $\hat{y} = y$ the machine got it right, when $\hat{y} \neq y$ it got it wrong. All of training is about making $\hat{y}$ equal to y as often as possible.

4. The bias, properly understood

The bias is the number students dismiss fastest, and yet it hides the most elegant idea in the perceptron. There are three ways of looking at it, and it is worth holding all three in your head.

First way: the bias is the threshold in disguise

Let us rewrite the condition "the perceptron says yes":

$$w_1x_1 + w_2x_2 + b \;\geq\; 0 \qquad \Longleftrightarrow \qquad w_1x_1 + w_2x_2 \;\geq\; -b$$

Read this way, the formula says: *take the weighted sum of the information and compare it with the threshold $-b$.* The bias is nothing but the threshold with its sign flipped and moved across. A very negative bias means a high threshold, that is, someone hard to convince.

Second way: the bias is the answer when nothing happens

Put $x_1 = 0$ and $x_2 = 0$ into the formula: what is left is $z = b$. So:

the bias is the score of the evening in which there is no friend
and the cinema is zero kilometres away, that is, your baseline urge to go out.

If $b > 0$ you are the sort who always goes out and it takes a good reason to keep you in. If $b < 0$ you are a homebody and it takes a good reason to get you out.

Third way, the most important: without a bias the boundary passes through the origin

This is the point that decides everything. Let us take the smallest possible example: a single input, the number of friends, and the rule "I go to the cinema if there are at least two friends".

BY HAND Try to build it *without* a bias, that is, with $z = w_1x_1$ and nothing else. With w_1 positive:

$$z = w_1 \cdot 0 = 0 \;\geq\; 0 \;\Rightarrow\; \hat{y} = 1$$

With zero friends the machine already says yes, and with more friends z grows further, so it always says yes. With w_1 negative the opposite happens and it always says no (except at $x_1 = 0$). **There is no w_1 whatsoever that implements "at least two friends".**

BY HAND Now put the bias back, with $w_1 = 1$ and $b = -2$:

$$z = x_1 - 2$$

x_1	$z = x_1 - 2$	$\hat{y}$	right?
0	-2	0	yes
1	-1	0	yes
2	0	1	yes
3	$+1$	1	yes

It works perfectly. The same holds in two dimensions: without a bias the separating line is **forced to pass through the origin** $(0, 0)$, because $w_1 \cdot 0 + w_2 \cdot 0 = 0$ always. The bias is what lets it come away from the origin and settle where it is needed.

The always-on input trick

Look again at Figure 1: the bias is drawn as an input that is always worth 1, with its own weight b. That is not a drawing convenience, it is a perfectly legitimate way of seeing it. If you set $x_0 = 1$ and call $w_0 = b$, the formula becomes

$$z = w_0x_0 + w_1x_1 + w_2x_2$$

and the bias disappears as a separate concept: it is simply the weight of a fake input that is always switched on. This trick is very convenient, because it means the weight update rule works for the bias too without writing a different one. In real libraries the bias is handled in exactly this way.

A note of honesty

In our cinema dataset there happens, by pure chance, to be a line through the origin that separates the data (it is the line $x_2 = 2x_1$). But the machine does not know that in advance, and in fact it will find a different line, one that does not touch the origin. In general the bias is needed, and removing it is one of the most common beginner's mistakes.

5. The first calculation: the machine just born

Here is the most important thing to understand in the whole course: **at the start the machine knows nothing**. Nobody has told it that distance is a negative thing or that friends help. The weights and the bias are chosen **at random**, as if we were rolling dice.

Suppose chance has handed us these starting values:

w_1 (friends)	w_2 (distance)	b (bias)
0.2	0.3	-0.1

Look carefully: chance has put $w_2 = +0.3$, that is, a *positive* weight on the distance. That means this newborn machine believes that *the further away the cinema is, the more I want to go*. It is nonsense, but the machine does not know that yet. Our job is to make it find out from the data.

We shall also use another number, the **learning rate** η (read "eta"), which decides how big the corrections are. We shall use $\eta = 0.1$: small, cautious corrections.

Row R1: $x = (1,\ 1)$**, true answer** $y = 1$

BY HAND Substitute the numbers into the formula and add up:

$$\begin{aligned} z &= w_1 x_1 + w_2 x_2 + b \\ &= 0.2 \cdot 1 \ + \ 0.3 \cdot 1 \ + \ (-0.1) \\ &= 0.2 + 0.3 - 0.1 \ = \ \mathbf{0.4} \end{aligned}$$

Apply the step: $0.4 \geq 0$, so $\hat{y} = 1$. The true answer was $y = 1$. **It got it right!** When the machine gets it right nothing is touched: the weights stay as they were.

Row R2: $x = (1,\ 3)$**, true answer** $y = 0$

BY HAND Same weights, different data:

$$z = 0.2 \cdot 1 + 0.3 \cdot 3 + (-0.1) \ = \ 0.2 + 0.9 - 0.1 \ = \ \mathbf{1.0}$$

The step says: $1.0 \geq 0$, so $\hat{y} = 1$, that is, "we go". But the true answer was $y = 0$: that Saturday we stayed in because one friend alone was not worth three kilometres. **It got it wrong**, and the culprit is plain to see: the term $0.3 \cdot 3 = 0.9$ on its own is worth more than everything else. It is the wrong weight on the distance.

Now we have to correct it.

6. The learning rule

Rosenblatt invented a correction rule of disarming simplicity. First you work out **the error**:

$$e \ = \ y - \hat{y}$$

There are only three possible cases, because y and $\hat{y}$ are only ever 0 or 1:

y	$\hat{y}$	$e = y - \hat{y}$	**what it means**
1	1	0	it got it right: touch nothing
0	0	0	it got it right: touch nothing
1	0	$+1$	it should have said yes and said no: raise the score
0	1	-1	it should have said no and said yes: lower the score

Then every weight is updated with this formula:

$$\boxed{w_i^{\text{new}} \;=\; w_i^{\text{old}} \;+\; \eta \cdot e \cdot x_i} \qquad \boxed{b^{\text{new}} \;=\; b^{\text{old}} \;+\; \eta \cdot e}$$

Why this formula works: read it out loud

The formula says: *correct each weight in proportion to how switched on that input was*. Look at the factor x_i at the end.
If $x_i = 0$, that input was off, it contributed nothing to the sum and therefore carries no blame: $\eta \cdot e \cdot 0 = 0$, the weight does not change. Quite right.
If x_i is large (say $x_2 = 6$ kilometres), that input pushed the result very hard and therefore carries a lot of blame: it gets corrected six times as much as an input worth 1.
The sign of e decides the direction. If $e = -1$ (we said yes and should not have) all the weights of the switched-on inputs go down, so that next time the score will be lower. If $e = +1$ they go up.
And the bias? The bias is updated with $\eta \cdot e$ and nothing else, without multiplying by anything. But it is the same formula: remember the always-on input trick? The bias has $x_0 = 1$, so $\eta \cdot e \cdot 1 = \eta \cdot e$.

Correcting row R2

We have $y = 0$ and $\hat{y} = 1$, so $e = 0 - 1 = -1$. The inputs were $x = (1,\, 3)$ and the rate is $\eta = 0.1$.

BY HAND One number at a time:

$$\begin{aligned} w_1 &= 0.2 + 0.1 \cdot (-1) \cdot 1 \;=\; 0.2 - 0.1 \;=\; \mathbf{0.1} \\ w_2 &= 0.3 + 0.1 \cdot (-1) \cdot 3 \;=\; 0.3 - 0.3 \;=\; \mathbf{0.0} \\ b &= -0.1 + 0.1 \cdot (-1) \;=\; \mathbf{-0.2} \end{aligned}$$

Look at what happened to w_2: it was 0.3 and it collapsed to 0.0 in a single move, because the distance was 3 and so it took three times the correction the friends did. The machine has just learnt its first lesson about the world: *distance does not count in the positive direction*.

Row R3: $x = (2,\, 3)$, true answer $y = 1$

Careful: the weights are now the *new* ones, that is $w = (0.1,\ 0.0)$ and $b = -0.2$. The perceptron learns row by row, it does not wait until the end.

BY HAND

$$z = 0.1 \cdot 2 + 0.0 \cdot 3 + (-0.2) \;=\; 0.2 + 0 - 0.2 \;=\; \mathbf{0.0}$$

$0.0 \geq 0$, so $\hat{y} = 1$: correct, it really was $y = 1$. No correction. Note that here the machine got it right by a whisker, sitting exactly on the edge.

Rows R4 and R5

BY HAND R4 is $x = (0,\, 3)$ with $y = 0$:

$$z = 0.1 \cdot 0 + 0.0 \cdot 3 - 0.2 \;=\; \mathbf{-0.2} \quad \Rightarrow \quad \hat{y} = 0 \quad \text{correct}$$

R5 is $x = (3,\, 5)$ with $y = 1$:

$$z = 0.1 \cdot 3 + 0.0 \cdot 5 - 0.2 \;=\; 0.3 - 0.2 \;=\; \mathbf{+0.1} \quad \Rightarrow \quad \hat{y} = 1 \quad \text{correct}$$

Row R6: the moment the machine understands

BY HAND R6 is $x = (2,\, 5)$ with $y = 0$:

$$z = 0.1 \cdot 2 + 0.0 \cdot 5 - 0.2 \;=\; 0.2 - 0.2 \;=\; \mathbf{0.0} \quad \Rightarrow \quad \hat{y} = 1$$

But $y = 0$: wrong, $e = -1$. Let us correct:

$$w_1 = 0.1 + 0.1 \cdot (-1) \cdot 2 \;=\; 0.1 - 0.2 \;=\; \mathbf{-0.1}$$
$$w_2 = 0.0 + 0.1 \cdot (-1) \cdot 5 \;=\; 0.0 - 0.5 \;=\; \mathbf{-0.5}$$
$$b = -0.2 - 0.1 \;=\; \mathbf{-0.3}$$

The weight of the distance has become negative. From now on every extra kilometre lowers the score. The machine has understood on its own, just by looking at the data, that a distant cinema is off-putting. Nobody told it.

It has overdone it, though: now the weight of the friends is negative too, which is absurd. The rows that follow will make it correct that.

7. The whole of epoch 1

One complete pass over all ten rows is called an **epoch**. Here is epoch 1 in full. You have already done the first six rows: check that they match, then carry on from R7 to R10.

	input			before						after		
row	x_1	x_2	y	w_1	w_2	b	z	$\hat{y}$	e	w_1	w_2	b
R1	1	1	1	0.2	0.3	−0.1	+0.4	1	0	*no change*		
R2	1	3	0	0.2	0.3	−0.1	+1.0	1	−1	0.1	0.0	−0.2
R3	2	3	1	0.1	0.0	−0.2	0.0	1	0	*no change*		
R4	0	3	0	0.1	0.0	−0.2	−0.2	0	0	*no change*		
R5	3	5	1	0.1	0.0	−0.2	+0.1	1	0	*no change*		
R6	2	5	0	0.1	0.0	−0.2	0.0	1	−1	−0.1	−0.5	−0.3
R7	4	6	1	−0.1	−0.5	−0.3	−3.7	0	+1	0.3	0.1	−0.2
R8	3	7	0	0.3	0.1	−0.2	+1.4	1	−1	0.0	−0.6	−0.3
R9	0	1	0	0.0	−0.6	−0.3	−0.9	0	0	*no change*		
R10	4	7	1	0.0	−0.6	−0.3	−4.5	0	+1	0.4	0.1	−0.2

Epoch 1 in summary: 5 errors out of 10 rows. Weights at the end: $w = (0.4,\ 0.1)$ and $b = -0.2$.

Notice the see-saw of the distance weight: $0.3 \to 0.0 \to -0.5 \to +0.1 \to -0.6 \to +0.1$. Learning is not a straight line: every row pulls the weights its own way and it takes a while before a compromise is found that suits all ten Saturdays.

Epoch 2

We start again from row R1, but with the weights we reached at the end of epoch 1.

	input			before						after		
row	x_1	x_2	y	w_1	w_2	b	z	$\hat{y}$	e	w_1	w_2	b
R1	1	1	1	0.4	0.1	−0.2	+0.3	1	0	*no change*		
R2	1	3	0	0.4	0.1	−0.2	+0.5	1	−1	0.3	−0.2	−0.3
R3	2	3	1	0.3	−0.2	−0.3	−0.3	0	+1	0.5	0.1	−0.2
R4	0	3	0	0.5	0.1	−0.2	+0.1	1	−1	0.5	−0.2	−0.3
R5	3	5	1	0.5	−0.2	−0.3	+0.2	1	0	*no change*		
R6	2	5	0	0.5	−0.2	−0.3	−0.3	0	0	*no change*		
R7	4	6	1	0.5	−0.2	−0.3	+0.5	1	0	*no change*		
R8	3	7	0	0.5	−0.2	−0.3	−0.2	0	0	*no change*		
R9	0	1	0	0.5	−0.2	−0.3	−0.5	0	0	*no change*		
R10	4	7	1	0.5	−0.2	−0.3	+0.3	1	0	*no change*		

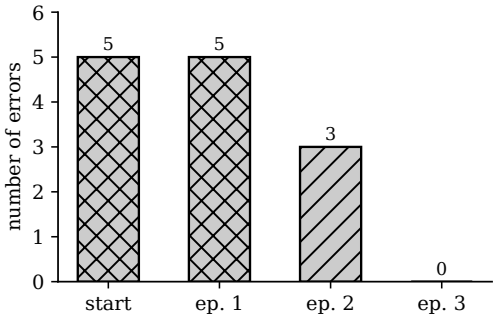
Epoch 2 in summary: 3 errors, all in the first four rows. From R5 onwards it gets nothing wrong. Final weights: $w = (0.5,\ -0.2)$, $b = -0.3$.

Epoch 3: the machine has learnt

In epoch 3 something rather beautiful happens: the machine goes over all ten rows and never gets one wrong. Check it yourself, row by row, with $w = (0.5,\ -0.2)$ and $b = -0.3$.

row	x_1	x_2	y	**the calculation**	z	$\hat{y}$
R1	1	1	1	$0.5 - 0.2 - 0.3$	0.0	1
R2	1	3	0	$0.5 - 0.6 - 0.3$	-0.4	0
R3	2	3	1	$1.0 - 0.6 - 0.3$	$+0.1$	1
R4	0	3	0	$0.0 - 0.6 - 0.3$	-0.9	0
R5	3	5	1	$1.5 - 1.0 - 0.3$	$+0.2$	1
R6	2	5	0	$1.0 - 1.0 - 0.3$	-0.3	0
R7	4	6	1	$2.0 - 1.2 - 0.3$	$+0.5$	1
R8	3	7	0	$1.5 - 1.4 - 0.3$	-0.2	0
R9	0	1	0	$0.0 - 0.2 - 0.3$	-0.5	0
R10	4	7	1	$2.0 - 1.4 - 0.3$	$+0.3$	1

Ten rows out of ten correct. No error means no correction, and no correction means the weights will never change again: training is over. We say the algorithm has **converged**.



The learning curve. The errors fall from 5 to 3 to 0. In three passes over the data the machine has gone from understanding nothing to understanding everything. A graph like this is called a *learning curve* and in every neural network project it is the first thing you look at: if it does not go down, something is wrong.

What rule has it learnt?

The final weights are $w_1 = 0.5$, $w_2 = -0.2$, $b = -0.3$. We can now read them the way you read a sentence:

- every **friend** adds 0.5 to the score;
- every **kilometre** takes away 0.2: the sign has turned negative, as it had to;
- the **bias** is negative (-0.3): we are a slightly lazy lot, and if there is no particular reason we stay in.

Let us do the exchange rate: how many kilometres is a friend worth? Just ask how many kilometres are needed to cancel out 0.5 of score, that is $0.5 \div 0.2 = 2.5$. So the machine has concluded that **every friend is worth two and a half kilometres of travel**. Nobody ever wrote that rule down: it extracted it on its own from ten examples. This, in miniature, is the whole of how neural networks work.

8. The line: where it comes from and how it moves

This is the central section of these notes. So far we have done sums; now let us look at them with our eyes and understand what those sums are drawing.

8.1 Why the boundary is a straight line

The perceptron changes its mind at one point only: when z goes from negative to positive. The border between the two answers is therefore the set of points where z is exactly zero:

$$w_1 x_1 + w_2 x_2 + b \;=\; 0$$

Now look carefully at that expression. The unknowns x_1 and x_2 appear only to the first power, multiplied by constants, and added together. It is the first degree equation in two unknowns you studied at school: its solution set is a **straight line**. This is not a choice made by whoever designed the perceptron, it is an unavoidable consequence of the fact that the perceptron can only multiply and add.

8.2 Putting it into explicit form

The equation $w_1 x_1 + w_2 x_2 + b = 0$ is in so-called *implicit form*. To draw it, it is convenient to isolate x_2, that is, to put it into the form $x_2 = m\,x_1 + q$ you already know how to handle. Solving for x_1 instead changes nothing, it is the same thing.

BY HAND Follow the three steps with our final weights $w_1 = 0.5$, $w_2 = -0.2$, $b = -0.3$:

$0.5\,x_1 - 0.2\,x_2 - 0.3 = 0$	the starting equation
$-0.2\,x_2 = -0.5\,x_1 + 0.3$	move the terms without x_2 to the right
$x_2 = \dfrac{-0.5\,x_1 + 0.3}{-0.2}$	divide everything by -0.2
$x_2 = 2.5\,x_1 - 1.5$	simplify

In general, repeating the same steps with letters:

$$\boxed{x_2 \;=\; \underbrace{\left(-\frac{w_1}{w_2}\right)}_{\text{slope } m} x_1 \;+\; \underbrace{\left(-\frac{b}{w_2}\right)}_{\text{intercept } q}}$$

BY HAND Check: $-\dfrac{w_1}{w_2} = -\dfrac{0.5}{-0.2} = +2.5$ and $-\dfrac{b}{w_2} = -\dfrac{-0.3}{-0.2} = -1.5$. They match.

8.3 What slope and intercept actually mean

- The **slope** $m = 2.5$ says how much the line rises when you move 1 to the right. In our problem the horizontal axis is friends and the vertical axis is kilometres, so $m = 2.5$ means exactly *each extra friend makes me accept 2.5 more kilometres of travel*. It is the same number we found by dividing 0.5 by 0.2, and that is no coincidence: it is the same division.
- The **intercept** $q = -1.5$ is the point where the line cuts the vertical axis, that is, where the threshold would sit with zero friends. It is negative, which means that with zero friends no distance is acceptable, not even zero kilometres. That is consistent with row R9, where on my own I stayed in even at one kilometre.

8.4 Which side is a point on: the test that decides everything

Given the line, how do I know whether a point lies in the "we go" region? No geometry is needed: **just substitute the coordinates of the point into the formula for z and look at the sign**.

BY HAND Take point R7, four friends and six kilometres:

$$z = 0.5 \cdot 4 - 0.2 \cdot 6 - 0.3 = 2.0 - 1.2 - 0.3 = +0.5 \ > \ 0$$

Positive sign: it lies in the "we go" region. And indeed $y = 1$.

Take R8, three friends and seven kilometres:

$$z = 1.5 - 1.4 - 0.3 = -0.2 \ < \ 0$$

Negative sign: "we stay" region. And indeed $y = 0$.

There is one question that must be asked, though: *is the "we go" region above or below the line?* The answer depends on the sign of w_2, and it is a point that confuses almost everybody. Let us go through the algebra again:

$$w_1 x_1 + w_2 x_2 + b \geq 0 \qquad \Longrightarrow \qquad w_2 x_2 \geq -w_1 x_1 - b$$

Now, to isolate x_2 I have to divide by w_2, and in an inequality **dividing by a negative number flips the direction**:

$$\text{if } w_2 > 0: \quad x_2 \geq m x_1 + q \quad \text{("we go" is ABOVE)}$$

$$\text{if } w_2 < 0: \quad x_2 \leq m x_1 + q \quad \text{("we go" is BELOW)}$$

In our case $w_2 = -0.2 < 0$, so the "we go" region is *below* the line. That makes sense: below means little distance.

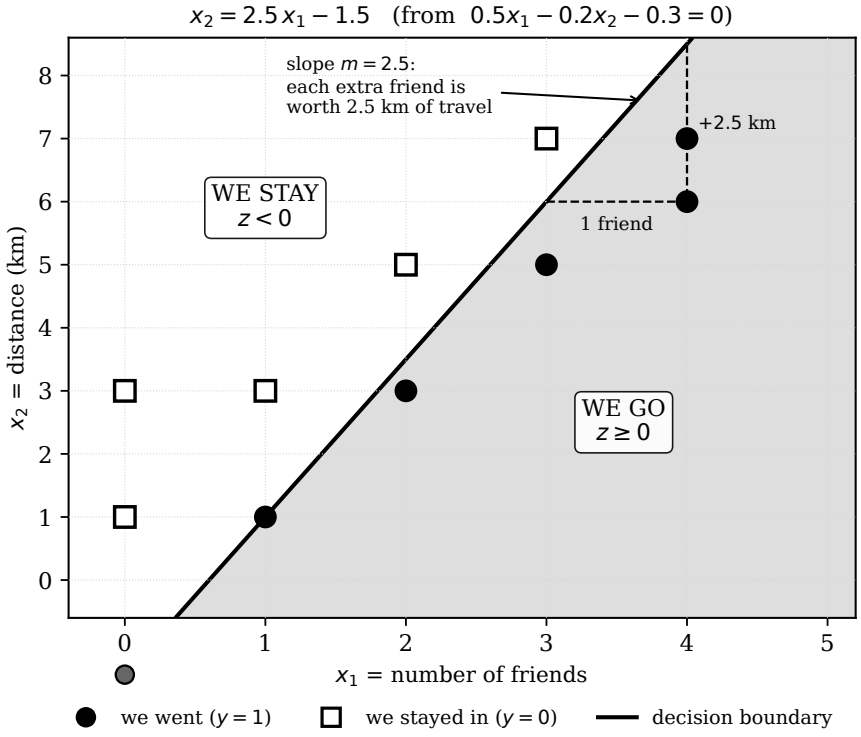


Figure 3: The line the perceptron learnt, with all its parts. The grey region is where $z \geq 0$, that is, where the machine answers "we go". Every black circle is inside it, every white square is outside.

The limiting case: $w_2 = 0$

If w_2 is exactly zero you cannot divide and the explicit form does not exist. What happens geometrically? The equation becomes $w_1 x_1 + b = 0$, that is, $x_1 = -b/w_1$: a **vertical line**. It is perfectly legitimate, it simply cannot be written in the form $x_2 = mx_1 + q$, exactly as with the lines you studied at school. In the epoch 1 table, just after row R2, the weights are $w = (0.1,\ 0.0)$: at that precise moment the boundary of our perceptron was the vertical line $x_1 = 2$, that is, the machine was deciding purely on the number of friends, ignoring distance altogether.

8.5 What each of the three numbers does

Now the point that makes everything clear: the three numbers w_1, w_2, b do not do the same thing to the line.

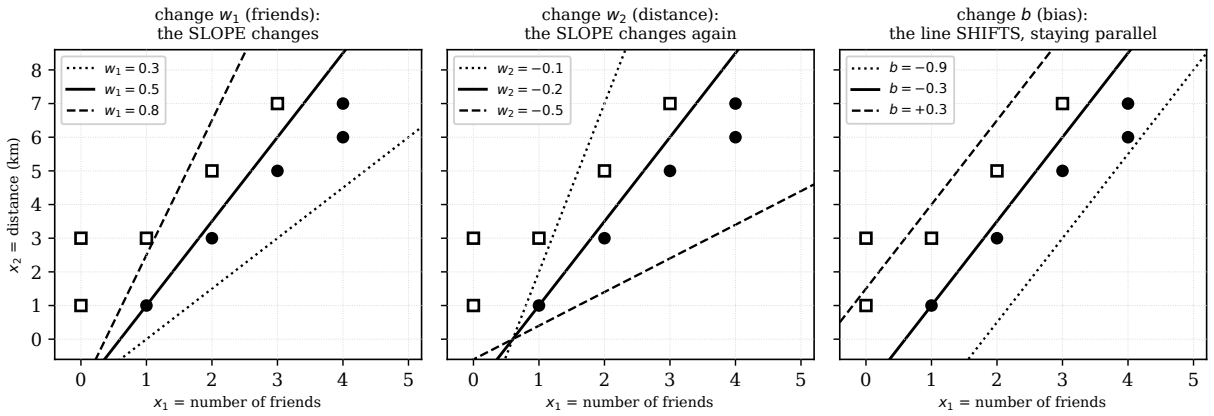


Figure 4: The role of the three numbers. Changing w_1 or w_2 makes the line **rotate**, because the ratio $-w_1/w_2$, which is the slope, changes. Changing the bias b makes the line **shift**, staying parallel to itself, because b appears only in the intercept $-b/w_2$.

if I change	**the line**	**why**
w_1 (friends)	rotates	appears only in the slope $-w_1/w_2$
w_2 (distance)	rotates, and if it changes sign the "we go" side flips too	appears both in the slope and in the intercept
b (bias)	shifts, staying parallel	appears only in the intercept $-b/w_2$

BY HAND Check the bias case. With $w = (0.5,\ -0.2)$ and $b = -0.9$ the intercept becomes $q = -(-0.9)/(-0.2) = -4.5$, while the slope stays at 2.5. The line is the same, moved down by 3. A more negative bias does indeed mean a lazier person, and therefore a smaller "we go" region.

8.6 How the line moves epoch after epoch

Now we can reread the whole of training as a film: at every correction of the weights the line moves.

There are two lessons in this figure, both important.

The first is that **learning is not monotonic**. In the third panel the machine is momentarily worse off than in the second. The perceptron always corrects by looking at one row at a time and has no idea how it is doing overall; it is like straightening a crooked picture while looking at it from one corner of the room only. Rosenblatt's theorem guarantees that it gets there in the end, not that it improves at every step.

The second is that **the sign of w_2 decides which side the "we go" region is on**. The moment w_2 goes from positive to negative is the moment the machine really understands the problem, and in the picture you see it as the grey region flipping over.

8.7 Another way of looking at the same thing

There is an even more direct picture: instead of points in the plane, let us draw on a single line the ten values of the score z computed with the final weights.

This is, in one sentence, everything a perceptron does: **it squashes a complicated situation into a single number and then looks at whether that number is positive or negative**. All the rest of these notes will be about making that squashing cleverer.

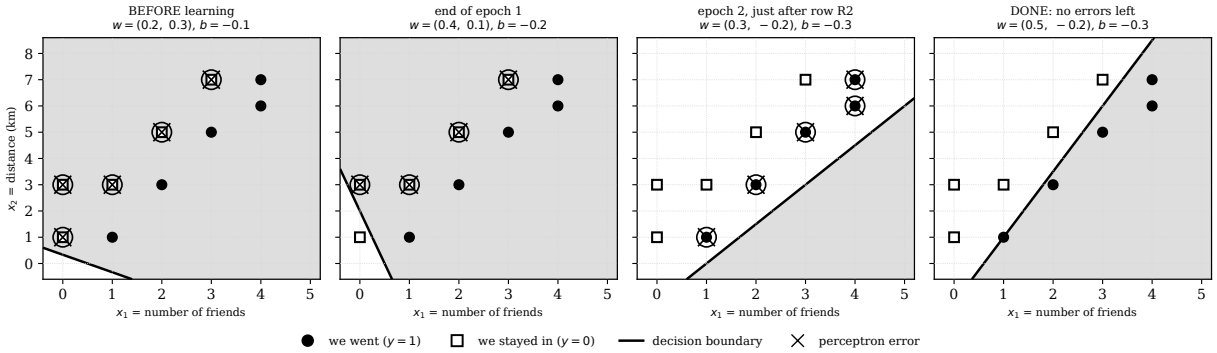


Figure 5: The line during training. **First panel**, the random weights: w_2 is positive, so the "we go" region is above and covers almost the whole picture; the machine says yes to everybody and gets wrong all five Saturdays on which we stayed in. **Second**, end of epoch 1: the line has straightened up but w_2 is still positive, so the wrong side is still above. **Third**, epoch 2 just after row R2: w_2 has just turned negative and *the grey region flips underneath*; the machine has overcorrected and now says no to everybody. **Fourth**, a few rows later it settles: no errors.

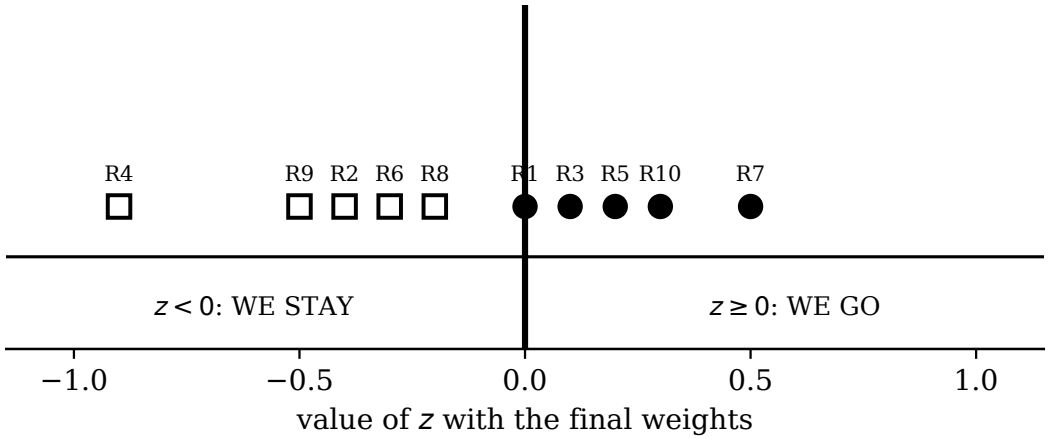


Figure 6: The ten Saturdays ordered by score. The thick vertical line is the threshold $z = 0$. Every black circle is to the right of it or exactly on it, every white square is to the left.

9. What if there are more than two inputs?

The straight line is a convenience of the two-input case. Let us see what happens in general, because we shall need it shortly.

inputs	**equation of $z = 0$**	**the boundary is**	**can you draw it?**
1	$w_1 x_1 + b = 0$	a point on the number line	yes
2	$w_1 x_1 + w_2 x_2 + b = 0$	a line in the plane	yes
3	$w_1 x_1 + w_2 x_2 + w_3 x_3 + b = 0$	a plane in space	with difficulty
n	$w_1 x_1 + \cdots + w_n x_n + b = 0$	a *hyperplane*	no, but the sums are identical

The word *hyperplane* sounds frightening, but there is nothing mysterious about it: it is just the name given to the same equation when there are many letters. The calculation to be done is still "multiply, add, look at the sign", identical to the one you did by hand a few pages ago. The fact that you cannot draw it is your problem, not the perceptron's.

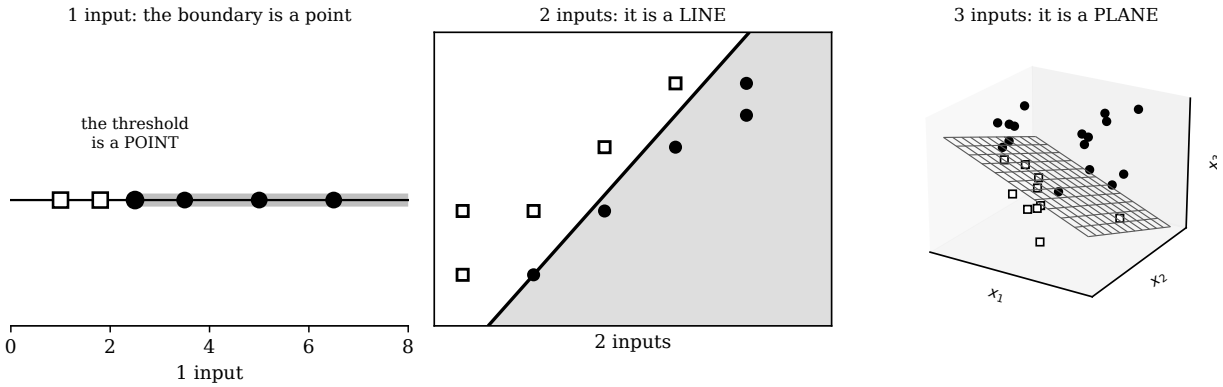


Figure 7: The same formula, in different dimensions. The boundary is always "the flat object with one dimension fewer than the space you live in".

The thing to take away

A perceptron, whatever the number of inputs, can do one thing only: cut space in two with a straight cut. Everything that comes later in these notes, hidden layers, backpropagation, deep learning, exists to get past that limit.

10. The moment of truth: validation and test

We have checked that the machine answers well on the ten Saturdays we made it study, but that is like giving a student a test with the very same questions they saw the day before. It proves nothing: they might have learnt them by heart.

That is why data is always split into **three separate groups**.

group	what it is for	the school analogy
training	the machine learns on it, changing the weights	the exercises done in class with the teacher
validation	you check whether it works outside, and you settle the details (how many epochs, what η should be)	the mock exam, which can be repeated
test	used **once only**, at the end, and nothing is touched afterwards	the final examination

The golden rule is: **you never train on the test data**. If you peek at the test and then change something, the test is burnt and no longer tells you the truth.

Validation: three Saturdays never seen before

BY HAND Work out the three scores yourself with $w = (0.5,\ -0.2)$ and $b = -0.3$ before reading the z column. The weights **are not touched any more**.

	x_1	x_2	the calculation	z	$\hat{y}$	true y	outcome
V1	2	2	$1.0 - 0.4 - 0.3$	$+0.3$	1	1	correct
V2	1	4	$0.5 - 0.8 - 0.3$	-0.6	0	0	correct
V3	3	4	$1.5 - 0.8 - 0.3$	$+0.4$	1	1	correct

Three out of three, that is 100% **accuracy**. Accuracy is simply the number of right answers divided by the number of questions.

In the training set there was no Saturday identical to any of these. Answering them correctly all the same is called **generalising**: being able to tackle a new question by putting together what you understood from other questions. It is the only thing that really counts in machine learning.

Test: the final examination

BY HAND The last four Saturdays, the ones that decide the mark.

	x_1	x_2	the calculation	z	$\hat{y}$	true y	outcome
T1	4	3	$2.0 - 0.6 - 0.3$	$+1.1$	1	1	correct
T2	0	5	$0.0 - 1.0 - 0.3$	-1.3	0	0	correct
T3	2	6	$1.0 - 1.2 - 0.3$	-0.5	0	0	correct
T4	3	2	$1.5 - 0.4 - 0.3$	$+0.8$	1	1	correct

Four out of four. The perceptron has passed the examination. You can of course also let the machine decide without giving it the outcome at all, and indeed that is the whole reason for using one: you give it things to learn in the training set, you check that it is capable on the validation set, and then you use it to classify and predict in the real world.

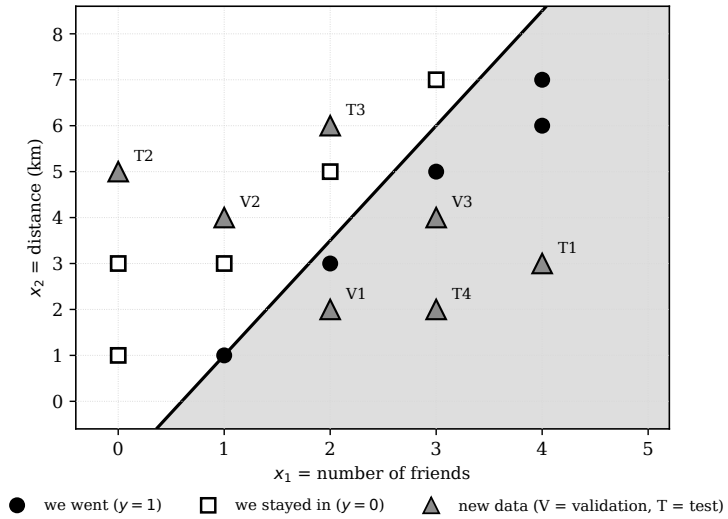


Figure 8: The seven new data points (grey triangles) placed on top of the line learnt from the training data alone. Nobody moved the line to let them in: the line was already there, and they happened to fall on the right side of it by themselves.

Careful: 100% is not always good news

Here we have little data, very tidy, built on purpose so that the rule really exists. In the real world things are messier and a perfect 100% is often the sign that something has gone wrong: usually that the test data has ended up in the training set by mistake. If a model of yours scores 100%, the first reaction should be suspicion, not celebration.

11. More neurons in the same layer

A single perceptron draws a single line. But if we put two of them side by side, each with its own weights and its own bias, we get **two lines** and we can combine them.

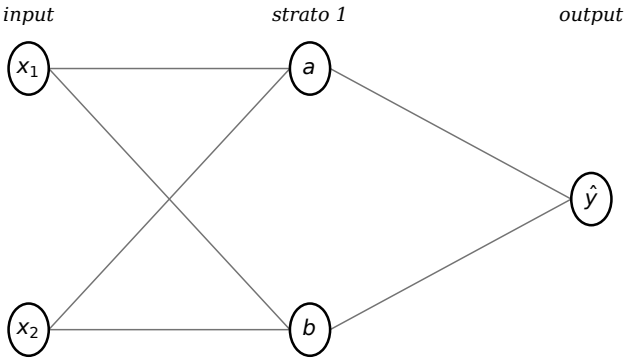


Figure 9: Two neurons in the same layer. The same inputs reach both, but the weights on the connections are different, so the two answers are different.

Let us change the cinema problem slightly, so that this is really needed. The new house rule is:

we go to the cinema if it is worth it given the number of friends,
but if the cinema is less than two kilometres away we walk
and stop at the pub, so we never actually get to the cinema.

Two conditions are needed at once and no single line can express them. So let us put in two neurons:

	w_1	w_2	b	**what it checks**
neuron a	0.5	-0.2	-0.3	"is it worth it" (the perceptron we trained)
neuron b	0.0	1.0	-2.0	"far enough to take the bus"

Neuron b has $w_1 = 0$: it ignores the friends completely and looks only at the distance. Its score is $z_b = x_2 - 2$, so it switches on when $x_2 \geq 2$.

BY HAND Fill in the table. For every row work out z_a and z_b first, then apply the step function to each.

x_1	x_2	$z_a = 0.5x_1 - 0.2x_2 - 0.3$	a	$z_b = x_2 - 2$	b	
1	1	$+0.0$	1	-1	0	too close
2	3	$+0.1$	1	$+1$	1	**we go**
0	3	-0.9	0	$+1$	1	no friends
3	5	$+0.2$	1	$+3$	1	**we go**
2	5	-0.3	0	$+3$	1	too few friends for 5 km
4	6	$+0.5$	1	$+4$	1	**we go**
1	4	-0.6	0	$+2$	1	one friend is not enough
4	3	$+1.1$	1	$+1$	1	**we go**

The two starting numbers have been **rewritten** into two new numbers, the pair (a, b), which describe the evening in a different language. This is called a **representation** and it is the most important thing a neural network does: not so much deciding, as *rewriting the problem into a form in which deciding becomes easy*.

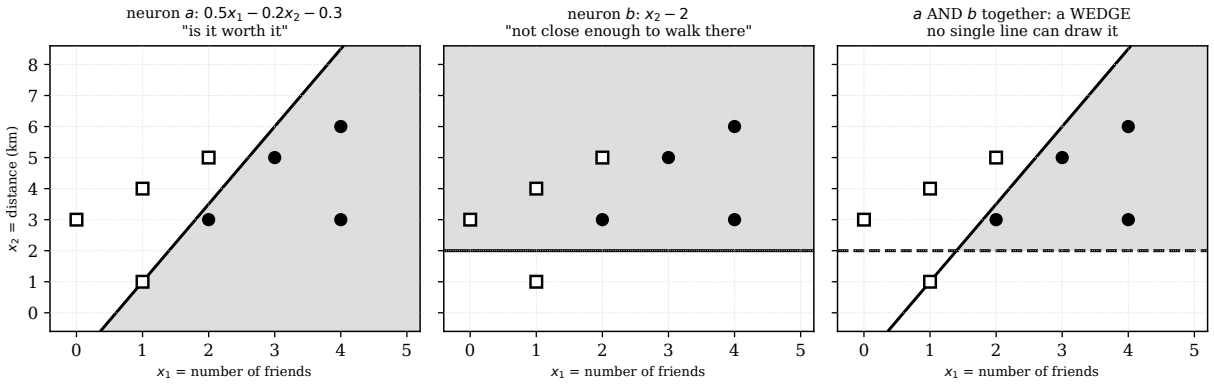


Figure 10: Two neurons, two lines. Each on its own cuts the plane in two (left and middle panels). But if we demand that *both* switch on, the good region is the intersection of the two, that is, a **wedge** (right panel). A wedge is not a half-plane and no single perceptron can draw it.

12. The second layer: what really changes

Now the two numbers a and b become the inputs of a third neuron, which will give the final answer. The calculation is identical to before, only the names change:

$$z_{\text{out}} = v_1 \cdot a + v_2 \cdot b + c \qquad \text{and then} \qquad \hat{y} = \text{step}(z_{\text{out}})$$

where v_1, v_2 are the weights of the second layer and c is its bias. The rule to remember is: **the output of one layer becomes the input of the next**. That is all. There is nothing else to know.

BY HAND Let us try with $v_1 = 1$, $v_2 = 1$, $c = -1.5$. For the Saturday $(2, 3)$, where we found $a = 1$ and $b = 1$:

$$z_{\text{out}} = 1 \cdot 1 + 1 \cdot 1 - 1.5 = +0.5 \;\Rightarrow\; \hat{y} = 1$$

And for the Saturday $(1, 1)$, where $a = 1$ and $b = 0$:

$$z_{\text{out}} = 1 \cdot 1 + 1 \cdot 0 - 1.5 = -0.5 \;\Rightarrow\; \hat{y} = 0$$

With these weights the output neuron answers 1 only if *both* the neurons below have switched on: it has implemented the logical **AND**, and it is precisely this AND that cuts out the wedge in the previous figure.

BY HAND Now change only the bias, from -1.5 to -0.5, and redo the two calculations. You will get $+1.5$ and $+0.5$, so 1 in both cases: now it is enough that *at least one* of the two switches on. That is the logical **OR**.

a	b	z with $c = -1.5$	**AND**	z with $c = -0.5$	**OR**
0	0	-1.5	0	-0.5	0
0	1	-0.5	0	$+0.5$	1
1	0	-0.5	0	$+0.5$	1
1	1	$+0.5$	1	$+1.5$	1

The right question: does the second layer draw a line too?

Yes, but **not in the picture you are looking at**. This is where almost everybody gets lost, so let us go slowly.

The output neuron receives a and b, not x_1 and x_2. So its line lives on a different sheet of paper: a sheet whose axes are a and b, not friends and kilometres. Let us call it the **hidden space**.

First layer	The passage	Second layer
draws lines on the sheet of the inputs (x_1, x_2), the one you see	every point of the input sheet is sent to a point of the (a, b) sheet	draws a line on the (a, b) sheet, which is a different sheet

The point is this: **a straight line on the hidden sheet, carried back to the original sheet, is no longer a straight line**. It becomes the edge of a wedge, of a strip, of a polygon. That is why two layers can do things a single layer cannot: not because they can draw curves, but because they draw straight lines on a sheet that the first layer has already redrawn.

In the next section we shall see this mechanism at work on the problem that made history.

13. The wall: the problem no straight line can solve

Let us change the house rule once more:

*We go to the cinema if it rains **or** if friends are around,*
*but if it rains **and** friends are around we stay in and play cards.*

With $x_1 =$ it rains (0 or 1) and $x_2 =$ friends are around (0 or 1):

x_1 **rains**	x_2 **friends**	y	
0	0	0	nothing doing, I stay in
0	1	1	friends are around: cinema
1	0	1	it rains and I am alone: cinema
1	1	0	it rains and friends are around: cards at home

This function is called **XOR**, that is, "one or the other but not both". It looks harmless. It was the grave of the perceptron.

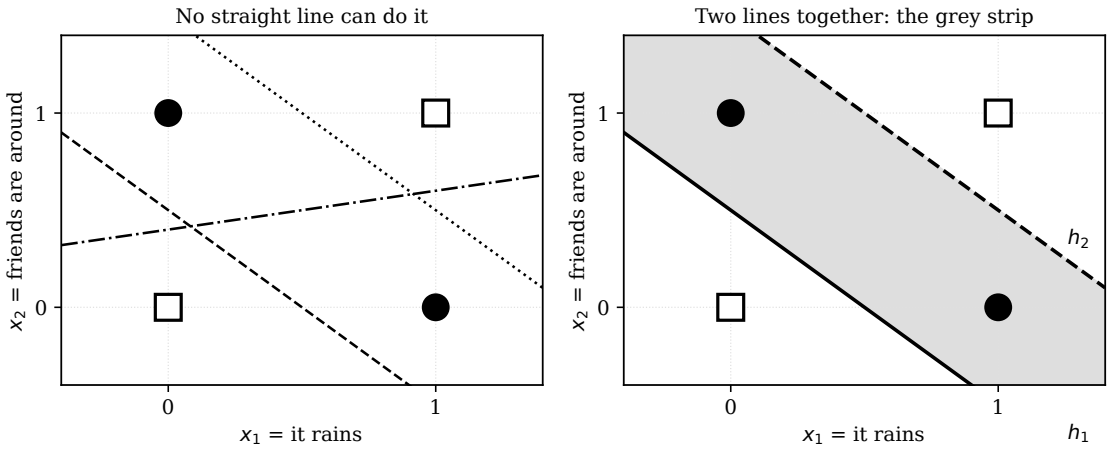


Figure 11: On the left: the four cases of XOR. The two black circles sit on one diagonal, the two white squares on the other. The three dashed lines are three attempts, and none of them works. **On the right**: with two lines, on the other hand, you can cut out the strip that contains exactly the two black circles.

13.1 The proof, in four lines

It is not that we failed to find the right line: the right line **does not exist**, and that is easy to prove. Suppose for contradiction that there are three numbers w_1, w_2 and b that work. Then all four of these conditions must hold:

from $(0,0) \to 0$:	$b < 0$	hence $-b > 0$
from $(1,0) \to 1$:	$w_1 + b \geq 0$	hence $w_1 \geq -b$
from $(0,1) \to 1$:	$w_2 + b \geq 0$	hence $w_2 \geq -b$
from $(1,1) \to 0$:	$w_1 + w_2 + b < 0$	

BY HAND Add the second and the third: $w_1 + w_2 \geq -2b$. Add b to both sides:

$$w_1 + w_2 + b \;\geq\; -2b + b \;=\; -b \;>\; 0$$

But the fourth condition demanded $w_1 + w_2 + b < 0$. A quantity cannot be both greater than and less than zero: **contradiction**. So there is no choice of weights that solves XOR with a single perceptron. It is not a matter of training it longer or being cleverer: it is mathematically impossible.

13.2 The solution, and why it works

Let us put two neurons in the first layer, one doing the OR and one doing the AND:

$$h_1 = \text{step}(x_1 + x_2 - 0.5) \quad \text{(the OR)}, \qquad h_2 = \text{step}(x_1 + x_2 - 1.5) \quad \text{(the AND)}$$

and then an output neuron:

$$\hat{y} = \text{step}(h_1 - h_2 - 0.5)$$

BY HAND Fill in the table yourself, row by row.

x_1	x_2	z_1	h_1	z_2	h_2	z_{out}	$\hat{y}$	**true** y
0	0	-0.5	0	-1.5	0	-0.5	0	0
0	1	$+0.5$	1	-0.5	0	$+0.5$	1	1
1	0	$+0.5$	1	-0.5	0	$+0.5$	1	1
1	1	$+1.5$	1	$+0.5$	1	-0.5	0	0

Four rows out of four. But *why* does it work? Look at the (h_1, h_2) column and compare it with the starting one:

starting point (x_1, x_2)	becomes (h_1, h_2)	answer wanted
$(0, 0)$	$(0, 0)$	0
$(0, 1)$	$(1, 0)$	1
$(1, 0)$	$(1, 0)$	1
$(1, 1)$	$(1, 1)$	0

The two points we wanted to separate from the others have ended up in the same place. Four starting points have become three arrival points, and among those three the problem is trivial: one straight line is enough.

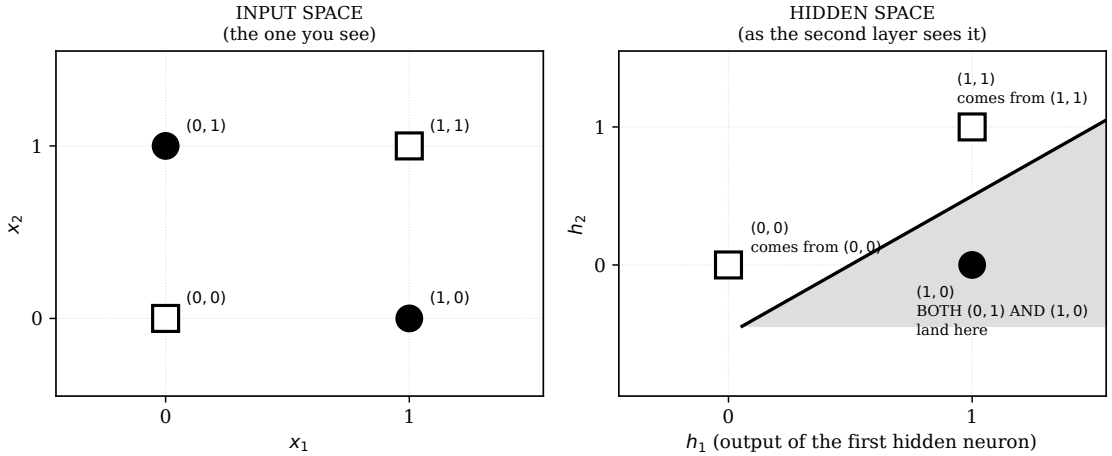


Figure 12: This is the most important figure in these notes. **On the left** the starting space, where the four points cannot be separated by a straight line. **On the right** the hidden space, after the first layer has rewritten every point. The points $(0, 1)$ and $(1, 0)$ have been pushed into the same place, and now the line $h_2 = h_1 - 0.5$ separates everything without any effort.

The sentence to remember for the rest of your life

A hidden layer does not learn to draw curves. **It learns to move the points** until the problem becomes simple enough to be solved with a straight line. Deep learning is that idea, repeated many times over.

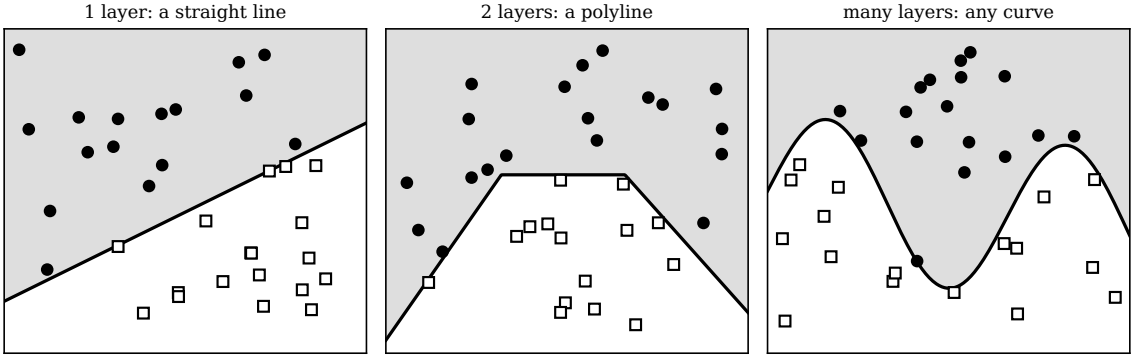


Figure 13: The result as seen from the original sheet. With one layer the boundary is a straight line; with two it becomes a polyline, because it is made of pieces of straight lines glued together; with many layers it can take any shape at all.

14. Rosenblatt, Minsky and twenty years of silence

It is worth telling how it really went, because it is one of the most instructive stories in twentieth century science.

In 1958 **Frank Rosenblatt** presents the perceptron publicly, and two years later he builds the physical version, the Mark I Perceptron: a machine the size of a wardrobe, full of wires and potentiometers, able to learn to recognise simple shapes. The newspapers go wild: the *New York Times* writes that the American Navy has created the embryo of a computer that will one day walk, talk, see, write and be conscious of itself.

Rosenblatt had also proved an important theorem, the **perceptron convergence theorem**: if the data can be separated by a straight line, then the algorithm you ran by hand *always finds one*, in a finite number of steps. It is a fine theorem. The trouble is the premise, that "if".

In 1969 **Marvin Minsky** and **Seymour Papert**, of MIT, publish the book *Perceptrons*, in which they prove with full rigour exactly what you proved yourself a few pages ago: a single-layer perceptron cannot do XOR, nor recognise whether a figure is connected, nor many other elementary things. The proof is impeccable and the tone is, at times, sarcastic about the excessive enthusiasm of those years.

Here, though, is the point that is often forgotten: Minsky and Papert **knew perfectly well** that with more layers the problem was solved, and they say so in the book. What they add is that nobody knew how to *train* a multi-layer network, and they declare themselves pessimistic that anyone would manage it.

The real obstacle, properly explained

The technical reason lies entirely in the step function, and you now have the tools to understand it fully.

With a single layer, when the machine gets something wrong you know exactly who is to blame: the inputs that were switched on. The formula $w_i \leftarrow w_i + \eta e x_i$ works precisely because x_i tells you how much blame each one carries.

With two layers you no longer know. If the final answer is wrong, is it the fault of neuron a? Of neuron b? Of the weight v_1? And by how much? The problem is called the **credit assignment problem** and with the step function it is insoluble, for a precise reason:

the step function is flat everywhere.

If you change a hidden weight by a hundredth, the hidden neuron almost certainly stays on the same step, its output stays identical, and the final answer does not change at all. The error tells

you "wrong" but does not tell you "wrong in this direction". You have no trail to follow. Trying all combinations of weights is impossible, because weights are continuous numbers.

The effect of the book was devastating. Funding for neural networks dried up almost everywhere and for them a seventeen-year dormancy began, lasting until backpropagation.

Be careful, though, not to confuse it with the **first AI winter**, which is a different thing: it starts five years later, in 1974, has a different cause (the Lighthill report commissioned by the British government) and concerns the whole field, not just networks. The two crises overlap in the middle but do not coincide, and in section 19.5 we set them out properly.

Rosenblatt was working on a solution to the problem but died shortly afterwards, in 1971, in a sailing accident, on his forty-third birthday. He took no part in the debate that followed and, above all, he never had time to show a solution. Fifty-one years later, at the age of ninety-four, he could have seen the fruit of his perceptron: ChatGPT.

The moral is not that Minsky was wrong: he was right about everything he proved. The moral is that an impossibility proof holds only under the assumptions it contains, and that changing a single assumption, in this case the step function, can reopen a road that seemed closed for ever.

On the other side, today's scientific community often blames Minsky for the collapse of funding for research on neural networks. Minsky, however, did not believe he had done any damage, but rather that he had brought scientific clarity to a field which, in his view, was dominated by sensational claims unsupported by evidence. It is worth pointing out that, for all the ferocity of the academic debate, Minsky and Rosenblatt had been schoolmates at the *Bronx High School of Science* and held each other in esteem. The dispute was purely scientific and epistemological; accordingly, Minsky never felt he owed any personal or moral apology to Rosenblatt's family.

15. A necessary interlude: what a derivative is

To get out of the winter we need one tool: the derivative. If you have not covered it at school yet, the following two pages will be enough to understand everything. And if they are not enough, there is ChatGPT, Claude, Gemini, DeepSeek or similar assistants that can help you study.

15.1 The idea: how much one thing changes if you move another

You have a quantity E (for us it will be the error) that depends on a number w (for us a weight). The **derivative of E with respect to w**, written $\dfrac{\partial E}{\partial w}$, answers one question only:

if I increase w by a little, by how much does E change?

If the answer is $+3$, it means that increasing w by a hundredth increases E by about three hundredths. If it is -3, E *decreases* by three hundredths. Geometrically it is the **slope of the tangent line** to the graph at that point.

15.2 How to compute it: first method, with a ruler

The most honest method is the one from the definition. Take a small step h, look at how much E changes, and divide:

$$\frac{E(w+h) - E(w)}{h}$$

This is called the **difference quotient**. Then make h smaller and see where the result goes.

BY HAND Let us do it on $E(w) = w^2$ at the point $w = 1.5$, where $E(1.5) = 2.25$:

h	$E(1.5+h)$	difference quotient
1	6.2500	$(6.2500-2.2500)/1 = 4.00$
0.5	4.0000	$(4.0000-2.2500)/0.5 = 3.50$
0.25	3.0625	$(3.0625-2.2500)/0.25 = 3.25$
0.1	2.5600	$(2.5600-2.2500)/0.1 = 3.10$
0.01	2.2801	$(2.2801-2.2500)/0.01 = 3.01$

The numbers close in on 3. And indeed $3 = 2 \cdot 1.5$: the derivative of w^2 is $2w$.

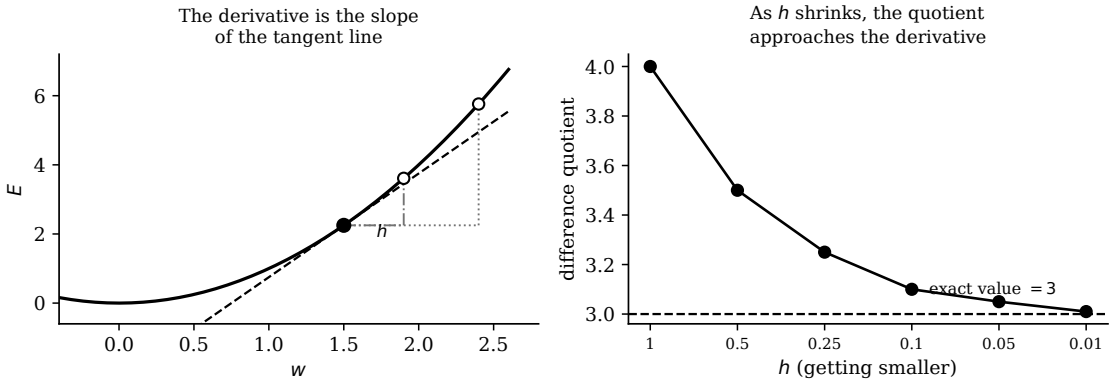


Figure 14: On the left: the derivative is the slope of the tangent line (dashed). The two small triangles show the difference quotient for two values of h: the smaller h is, the more the line joining the two points resembles the tangent. On the right: the table above, in graph form.

This method always works and is genuinely used on computers, to check that formulas written by hand are right. It is called *gradient checking* and we shall do it too in a moment.

15.3 How to compute it: second method, with the rules

In practice one never takes the limit: one learns five rules and applies them. These are they, and they are enough for the whole of these notes.

function	**derivative**	**example**
a constant k	0	the derivative of 7 is 0
w	1	
w^n	$n\,w^{n-1}$	the derivative of w^3 is $3w^2$
$k \cdot f(w)$	$k \cdot f'(w)$	the derivative of $5w^2$ is $10w$
$f(w)+g(w)$	$f'(w)+g'(w)$	the derivative of w^2+w is $2w+1$

BY HAND Check that the derivative of $E = \frac{1}{2}(y-\hat{y})^2$ with respect to $\hat{y}$, with y held fixed, is $-(y-\hat{y})$. Here are the steps: put $u = y-\hat{y}$; then $E = \frac{1}{2}u^2$ and by the power rule $\partial E/\partial u = u$. But when $\hat{y}$ grows by 1, u *falls* by 1, so $\partial u/\partial \hat{y} = -1$. Multiplying: $u \cdot (-1) = -(y-\hat{y})$.

That "multiplying" was not a trick. It was the single most important rule of all.

15.4 The chain rule, or the currency exchange

If E depends on u, and u depends on w, then

$$\frac{\partial E}{\partial w} = \frac{\partial E}{\partial u} \cdot \frac{\partial u}{\partial w}$$

The easiest way to remember it is to think of currency exchange. If one euro is worth 1.1 dollars, and one dollar is worth 150 yen, how many yen is one euro worth? You multiply: $1.1 \times 150 = 165$. Derivatives behave in exactly the same way: they are "exchange rates", that is, how much of one thing you get per unit of another, and they compose by multiplication.

And here is why it is the most important rule: **a neural network is a chain of functions**. The error depends on the output, which depends on the score, which depends on the weights. To find out how much a weight at the far end of the chain matters, you just multiply the exchange rates along the whole path. This, and nothing else, is backpropagation.

16. The gradient: the idea that changed everything

What exactly was missing? What was missing was a way of measuring **how much** you are wrong, not just *whether* you are wrong.

16.1 First step: throw the step function in the bin

Let us replace the step with a smooth curve, the **sigmoid**:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

What matters is the shape: it rises gently from 0 to 1, passing through 0.5 when $z = 0$. The output is no longer a blunt decision but a **degree of conviction**: 0.95 means "almost certainly yes", 0.5 means "I have no idea", 0.1 means "almost certainly no".

Its derivative has a delightfully memorable form:

$$\sigma'(z) = \sigma(z) \cdot \big(1 - \sigma(z)\big)$$

z	-3	-2	-1	0	1	2	3
$\sigma(z)$	0.05	0.12	0.27	0.50	0.73	0.88	0.95
$\sigma'(z)$	0.05	0.10	0.20	**0.25**	0.20	0.10	0.05

BY HAND Check one cell: $\sigma(1) = 0.73$, so $\sigma'(1) = 0.73 \cdot (1 - 0.73) = 0.73 \cdot 0.27 \approx 0.20$.

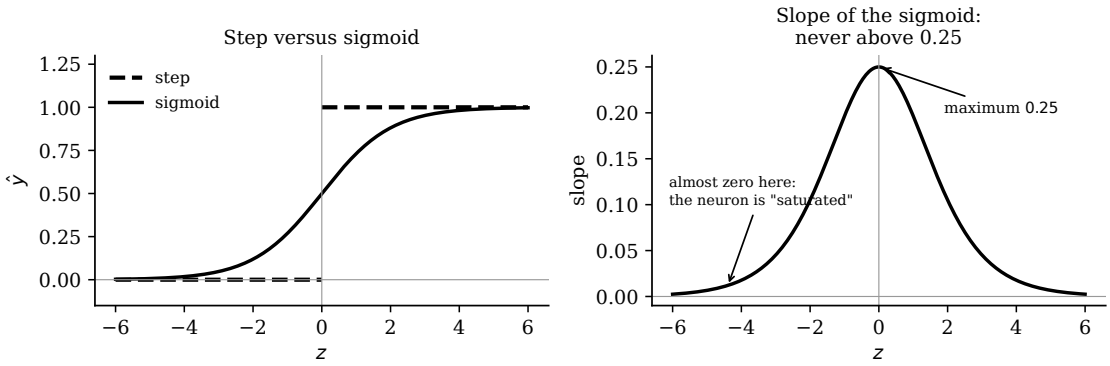


Figure 15: On the left the sigmoid compared with the step. On the right its slope, which is at most 0.25 and collapses to zero at the extremes. Keep that number 0.25 firmly in mind: three pages from now it will be the main character.

16.2 Second step: measure the error with a number

Let us define the error on a single example as

$$E = \tfrac{1}{2}\,(y - \hat{y})^2$$

We square it so that the error is always positive (it does not matter which way you are wrong) and because being very wrong weighs much more than being slightly wrong. The one half in front is there only to make the sums come out prettier when we differentiate.

Now E is a number that changes continuously: if I move a weight by a tiny amount, E changes by a tiny amount. **And that is exactly what we were missing.**

16.3 Third step: follow the slope

Imagine standing on a hill in fog, wanting to get down to the valley. You cannot see anything, but you can feel with your feet which way the ground slopes. You take a step in that direction, stop, feel again, take another step. Sooner or later you reach the bottom. The hill is the error E, your position is the weights, the slope under your feet is the **gradient**:

$$\boxed{w_i^{\text{new}} = w_i^{\text{old}} \; - \; \eta \cdot \frac{\partial E}{\partial w_i}}$$

There is a minus sign because we want the error to *go down*, so we move in the direction opposite to the uphill one.

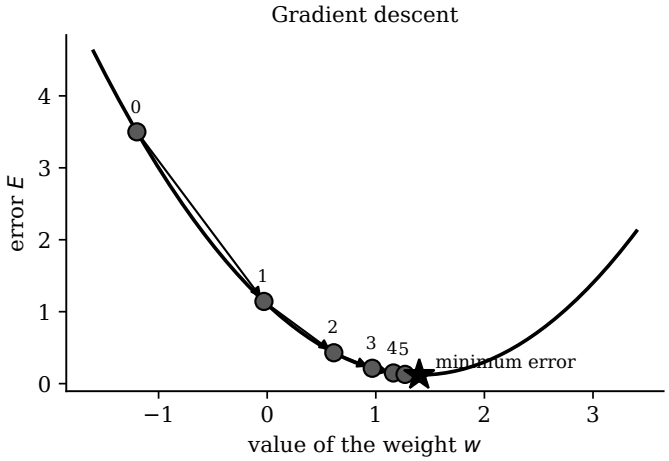


Figure 16: Gradient descent. The steps are large where the slope is steep and become tiny near the minimum: it is the automatic brake built into the method.

16.4 One neuron, one step, all by hand

Take the Saturday $x = (1,\, 2)$ with true answer $y = 1$, weights $w = (0.4,\; -0.2)$, bias $b = 0$ and $\eta = 1$.

BY HAND **Step 1, the weighted sum.**

$$z = 0.4 \cdot 1 + (-0.2) \cdot 2 + 0 = 0.4 - 0.4 = \mathbf{0}$$

Step 2, the sigmoid. $\sigma(0) = 0.5$ and $\sigma'(0) = 0.5 \cdot 0.5 = \mathbf{0.25}$. The machine answers $\hat{y} = 0.5$: completely undecided.

Step 3, the error. $E = \frac{1}{2}(1 - 0.5)^2 = \frac{1}{2} \cdot 0.25 = \mathbf{0.125}$

Step 4, the gradient via the chain rule. The chain is $w_1 \to z \to \hat{y} \to E$, so three exchange rates get multiplied:

$$\frac{\partial E}{\partial w_1} = \underbrace{\frac{\partial E}{\partial \hat{y}}}_{-(y-\hat{y})} \cdot \underbrace{\frac{\partial \hat{y}}{\partial z}}_{\sigma'(z)} \cdot \underbrace{\frac{\partial z}{\partial w_1}}_{x_1} = -(1 - 0.5) \cdot 0.25 \cdot 1 = \mathbf{-0.125}$$

It is worth giving a name to the piece that always recurs:

$$\delta \;=\; -(y - \hat{y}) \cdot \sigma'(z) \;=\; -0.125$$

and then all the other gradients follow immediately:

$$\frac{\partial E}{\partial w_2} = \delta \cdot x_2 = -0.125 \cdot 2 = -0.25, \qquad \frac{\partial E}{\partial b} = \delta \cdot 1 = -0.125$$

Step 5, the update. Remember the minus sign: subtracting a negative number means adding.

$$w_1 = 0.4 + 0.125 = \mathbf{0.525}, \quad w_2 = -0.2 + 0.25 = \mathbf{0.05}, \quad b = 0 + 0.125 = \mathbf{0.125}$$

Gradient checking: trust but verify

How do I know that -0.125 is right and that I have not got a sign wrong? I use the ruler method: I compute E for $w_1 = 0.4$ and for $w_1 = 0.4 + h$, and divide.

h	$E(0.4 + h)$	quotient
0.4	0.0805	-0.1112
0.1	0.1128	-0.1218
0.01	0.1238	-0.1247

The numbers run towards -0.125: the formula is right. This check is exactly what researchers do when they write a new network, and it unmasks nearly every sign error.

16.5 What changes compared with Rosenblatt

	perceptron rule (1958)	**gradient descent**
output	only 0 or 1	a number between 0 and 1
error	$+1$, -1 or 0	any number at all, even 0.043
when it corrects	only if it got it completely wrong	always, even when it is right by a narrow margin
how many layers	one only	as many as you like
guarantee	finds the line if one exists	always goes downhill, but may stop in a hollow

17. Backpropagation, step by step

Now the big piece: how a two-layer network is trained. We shall do it on a tiny but genuine network, with numbers chosen so that the sums come out exact.

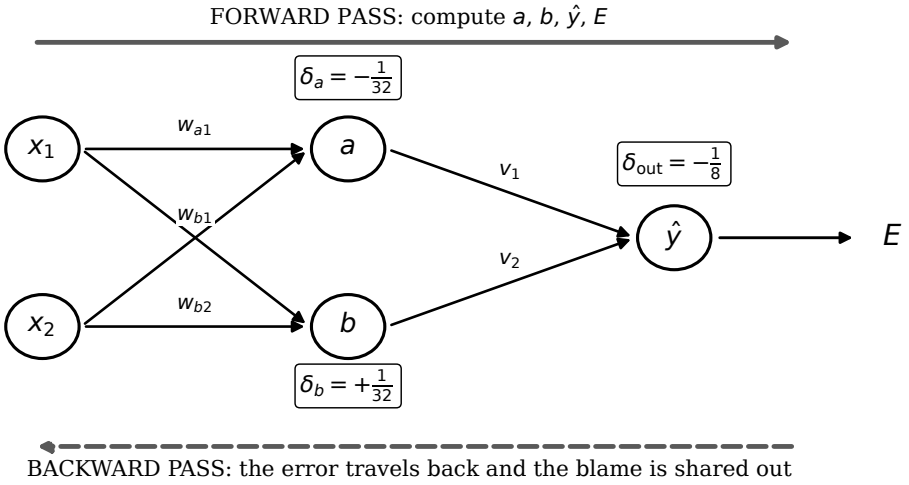


Figure 17: The network of the example and the two passes. First you go forwards to compute the answer and the error, then you come back to share out the blame.

The network has two inputs, two hidden neurons (a and b) and one output neuron. All of them use the sigmoid. The starting numbers:

	weight on x_1	**weight on x_2**	**bias**
neuron a	$w_{a1} = 0.5$	$w_{a2} = -0.25$	$b_a = 0$
neuron b	$w_{b1} = -0.5$	$w_{b2} = 0.25$	$b_b = 0$
	weight on a	**weight on b**	**bias**
output	$v_1 = 1$	$v_2 = -1$	$c = 0$

Example: $x = (1, 2)$ with true answer $y = 1$. Learning rate $\eta = 2$ (large, but it keeps the numbers pretty).

17.1 Phase 1: the forward pass

BY HAND Everything is computed from left to right.

$$z_a = 0.5 \cdot 1 + (-0.25) \cdot 2 + 0 = 0.5 - 0.5 = 0 \quad \Rightarrow \quad a = \sigma(0) = \tfrac{1}{2}$$
$$z_b = -0.5 \cdot 1 + 0.25 \cdot 2 + 0 = -0.5 + 0.5 = 0 \quad \Rightarrow \quad b = \sigma(0) = \tfrac{1}{2}$$
$$z_{\text{out}} = 1 \cdot \tfrac{1}{2} + (-1) \cdot \tfrac{1}{2} + 0 = 0 \quad \Rightarrow \quad \hat{y} = \sigma(0) = \tfrac{1}{2}$$

The error is $E = \frac{1}{2}\left(1 - \frac{1}{2}\right)^2 = \frac{1}{8} = 0.125$.

The three slopes we shall need are all equal to $\sigma'(0) = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$.

17.2 Phase 2: the backward pass

Now we come back from right to left. The question we answer for every neuron is always the same: *by how much would the error change if the score z of this neuron were slightly different?* Let us call that answer δ.

The output neuron.

$$\delta_{\text{out}} = -(y - \hat{y}) \cdot \sigma'(z_{\text{out}}) = -\left(1 - \tfrac{1}{2}\right) \cdot \tfrac{1}{4} = -\tfrac{1}{2} \cdot \tfrac{1}{4} = -\tfrac{1}{8} = \mathbf{-0.125}$$

From here the gradients of the last layer's weights come out immediately, because as always you multiply the δ by whatever was coming into the connection:

$$\frac{\partial E}{\partial v_1} = \delta_{\text{out}} \cdot a = -\tfrac{1}{8} \cdot \tfrac{1}{2} = -\tfrac{1}{16}, \qquad \frac{\partial E}{\partial v_2} = \delta_{\text{out}} \cdot b = -\tfrac{1}{16}, \qquad \frac{\partial E}{\partial c} = -\tfrac{1}{8}$$

The hidden neurons: this is where the magic happens. How much blame does neuron a carry? Reason it out as you would yourself: a influenced the error *only by passing through* the output neuron, and the channel it used was the weight v_1. So you take the δ of the one downstream, multiply it by the weight of the connection and then by your own slope. It is the chain rule again, the exchange rates multiplying:

$$\delta_a = \underbrace{\delta_{\text{out}}}_{\text{blame downstream}} \cdot \underbrace{v_1}_{\text{how much gets through the connection}} \cdot \underbrace{\sigma'(z_a)}_{\text{my own slope}} = -\tfrac{1}{8} \cdot 1 \cdot \tfrac{1}{4} = -\tfrac{1}{32} = \mathbf{-0.03125}$$

$$\delta_b = -\tfrac{1}{8} \cdot (-1) \cdot \tfrac{1}{4} = +\tfrac{1}{32} = \mathbf{+0.03125}$$

Note the sign of δ_b: it is the opposite, because the connection to the output has weight -1. Neuron b has to move in the opposite direction from a to achieve the same effect. The network works that out on its own, without anyone telling it.

And now the gradients of the first layer's weights, always with the same formula, "δ times whatever comes in":

$$\frac{\partial E}{\partial w_{a1}} = \delta_a \cdot x_1 = -\tfrac{1}{32} \cdot 1 = -\tfrac{1}{32}, \qquad \frac{\partial E}{\partial w_{a2}} = \delta_a \cdot x_2 = -\tfrac{1}{32} \cdot 2 = -\tfrac{1}{16}, \qquad \frac{\partial E}{\partial b_a} = -\tfrac{1}{32}$$

17.3 Phase 3: everything gets updated

BY HAND With $\eta = 2$, and remembering that the gradient is *subtracted*:

$$v_1 = 1 - 2 \cdot \left(-\tfrac{1}{16}\right) = 1 + \tfrac{1}{8} = \mathbf{1.125} \qquad v_2 = -1 + \tfrac{1}{8} = \mathbf{-0.875} \qquad c = 0 + \tfrac{1}{4} = \mathbf{0.25}$$
$$w_{a1} = 0.5 + \tfrac{1}{16} = \mathbf{0.5625} \qquad w_{a2} = -0.25 + \tfrac{1}{8} = \mathbf{-0.125} \qquad b_a = 0 + \tfrac{1}{16} = \mathbf{0.0625}$$
$$w_{b1} = -0.5 - \tfrac{1}{16} = \mathbf{-0.5625} \qquad w_{b2} = 0.25 - \tfrac{1}{8} = \mathbf{0.125} \qquad b_b = 0 - \tfrac{1}{16} = \mathbf{-0.0625}$$

17.4 Phase 4: did it work?

Let us redo the forward pass with the new weights:

$$z_a = 0.5625 - 0.25 + 0.0625 = 0.375 \qquad \Rightarrow a = \sigma(0.375) \approx 0.593$$
$$z_b = -0.5625 + 0.25 - 0.0625 = -0.375 \qquad \Rightarrow b = \sigma(-0.375) \approx 0.407$$
$$z_{\text{out}} = 1.125 \cdot 0.593 - 0.875 \cdot 0.407 + 0.25 \approx 0.560 \qquad \Rightarrow \hat{y} \approx 0.637$$

$$E = \tfrac{1}{2}(1 - 0.637)^2 \approx \mathbf{0.066}$$

It was 0.125, now it is 0.066: almost halved in a single step, and the answer has risen from 0.50 to 0.64, that is, towards the 1 we wanted.

The summary in three lines

1. **Forwards**: compute all the outputs up to the error.
2. **Backwards**: start from the last δ and propagate it leftwards, multiplying each time by the weight of the connection and by the slope of the neuron.
3. **Update**: the gradient of a weight is always *the δ of the neuron the connection arrives at, times the value the connection was carrying*.

That is all. A network with a hundred billion weights uses exactly these three lines.

18. The second obstacle: when the gradient vanishes

Backpropagation is made widely known in 1986 by **David Rumelhart**, **Geoffrey Hinton** and **Ronald Williams** in a paper in *Nature* (the idea had appeared earlier, for instance in Paul Werbos's 1974 thesis, but it is that paper that convinces the world). Minsky's problem is solved. Networks with two or three layers begin to work.

So why did deep learning arrive only twenty-five years later?

Because as soon as you try to add layers, a new problem jumps out, and its explanation is already in the sums you have just done by hand.

18.1 The calculation that explains everything

It concerns this step of backpropagation:

$$\delta_a = \delta_{\text{out}} \cdot v_1 \cdot \sigma'(z_a)$$

We got $-\frac{1}{8} \to -\frac{1}{32}$, that is, the signal was **reduced to a quarter** in crossing a single layer. That is no accident: it is the fault of σ', which as you saw in the table **never exceeds** 0.25.

BY HAND Now imagine repeating that step ten times, that is, having ten layers:

$$0.25^{10} = 0.00000095 \approx \frac{1}{1\,000\,000}$$

layers	fraction of gradient left	
1	0.25	a quarter
2	0.0625	a sixteenth
5	0.00098	a thousandth
10	0.00000095	a millionth
20	0.00000000000091	effectively zero

The first layers receive no signal at all any more. Their weights stay almost identical to the random starting ones, for ever. The network is deep only on paper: in practice only the last layers learn. This is called the **vanishing gradient problem**.

And it can go even worse the other way. If the weights are large, say each layer multiplies by 3 instead of by 0.25, then after twenty layers the factor is $3^{20} \approx 3.5$ billion: the gradient **explodes**, the weights shoot off to absurd values and training falls apart. That is the **exploding gradient**.

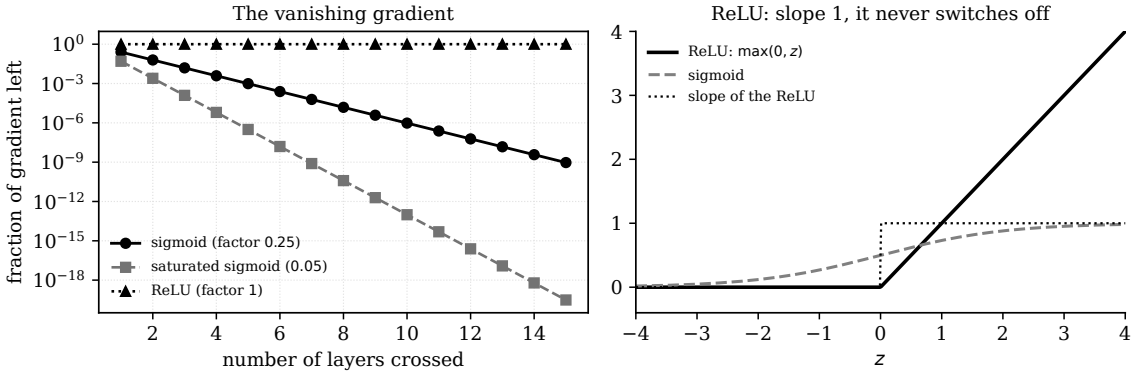


Figure 18: On the left, how much of the gradient is left after n layers (note the vertical scale: each tick is ten times the previous one). On the right, the ReLU function and its slope, which is 1 and therefore reduces nothing.

18.2 Who understood it and who solved it

The problem is identified precisely in 1991 by **Sepp Hochreiter** in his diploma thesis, and is analysed thoroughly in 1994 in a paper by **Yoshua Bengio**, Patrice Simard and Paolo Frasconi showing that learning long dependencies with gradient descent is intrinsically difficult. Bengio is a name worth remembering, because we shall meet him twice more in these pages.

The solutions arrived one at a time, and almost all of them are simple:

remedy	who and when	what it does
ReLU $\max(0, z)$	Glorot, Bordes and Bengio, 2011	the slope is 1 instead of 0.25: the signal is no longer reduced
clever initialisation	Glorot and Bengio 2010, then He 2015	the initial weights are given the right scale so that the signal neither explodes nor dies out
normalisation	Ioffe and Szegedy, 2015	the outputs of each layer are rescaled during training
residual connections	He et al., 2015	shortcuts are added that skip over layers, so the gradient has a straight road back

The ReLU deserves a moment's attention because it is the loveliest of them: it is the most stupid function imaginable, $\max(0, z)$, that is, "if the number is negative write zero, otherwise leave it alone". Its slope is 1 for $z > 0$, and 1 multiplied by itself a thousand times is still 1. The gravest problem in deep learning was solved in large part by a function that can be explained in half a line.

19. From the perceptron to deep learning

Now we can tell the whole story. And it is worth starting long before the perceptron, because the idea of a machine that calculates on our behalf is older than electricity.

Curiosity: the mechanical forerunners, when there was no electricity

1642, Blaise Pascal builds the *Pascaline* to help his father, a tax collector in Rouen: gears that add and subtract with automatic carries. About twenty of them will be built. Pascal is nineteen years old.
1804, Joseph-Marie Jacquard presents a loom driven by **punched cards**: where there is a hole a needle passes through, where there is none it stays put. Change the pack of cards and you change the pattern of the cloth without touching the machine. It is the first time a program is an object separate from the machine that runs it.
1837, Charles Babbage designs the *Analytical Engine*, which already has everything a computer has: a memory (he calls it the *store*), an arithmetic unit (the *mill*), conditional jumps and punched cards borrowed from Jacquard. He will never build it: it costs too much and the mechanical engineering of the day cannot hold the tolerances required.
1843, Ada Lovelace translates from the French a paper by the Italian Luigi Menabrea on Babbage's machine and adds *Notes* about three times as long as the original. Note G contains the procedure for making the machine compute the Bernoulli numbers: it is regarded as the first program in history, written for a machine that never existed. Hers too is the loveliest image in all of computing: the Analytical Engine *weaves* algebraic patterns as Jacquard's loom weaves flowers and leaves.
Ada Lovelace was English, daughter of the poet Byron, and built no machine at all: the French connection is that the paper she translated was written in French, and that the two genuine mechanical builders of the story, Pascal and Jacquard, were French.

19.1 1843: the objection we have carried with us ever since

In those same Notes, Ada Lovelace writes a sentence that still weighs on us today. In essence it says: *the Analytical Engine has no pretensions to produce anything new; it can only do what we know how to order it to do.*

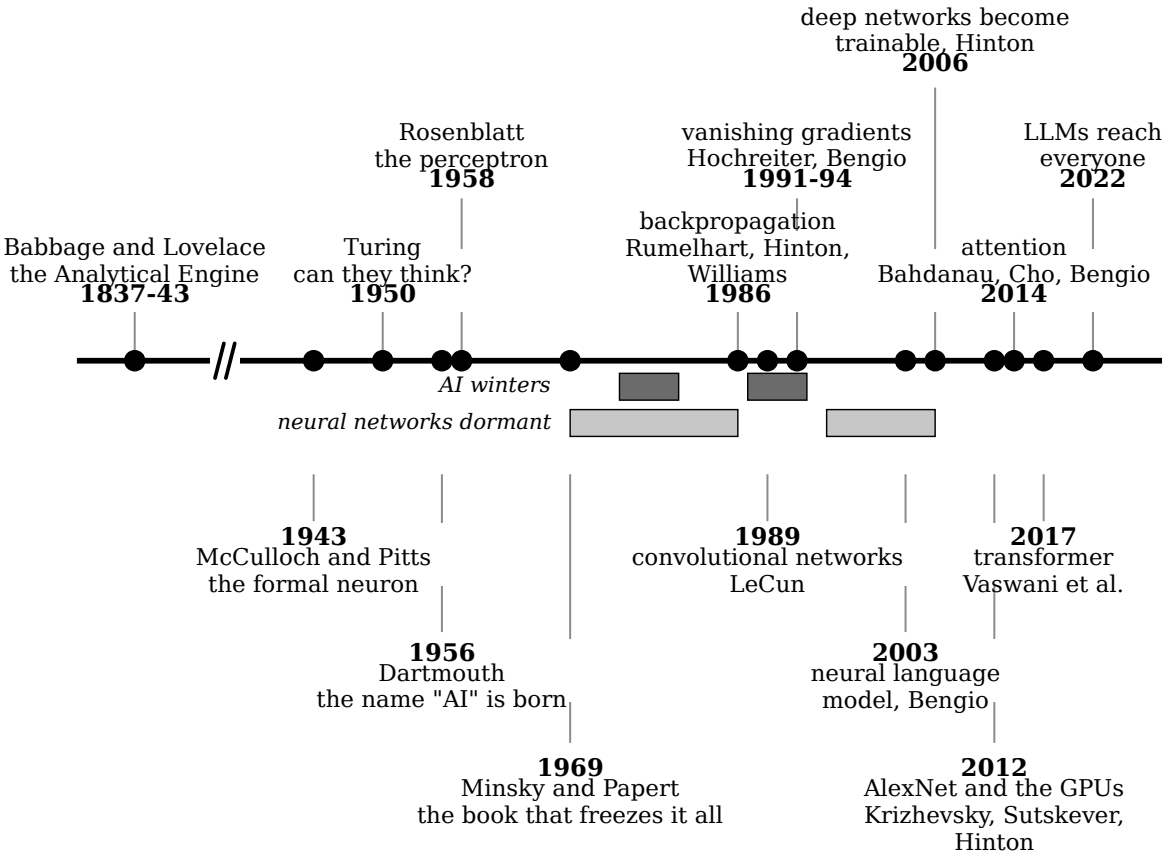


Figure 19: Almost two centuries in a single line. The mark // on the line is an *axis break*: between 1843 and 1943 the scale is compressed by about eight times, because in that century nothing happens that matters for this story. No event falls inside the cut stretch, and that is precisely the condition that makes it legitimate to cut it: the distances between the points actually drawn remain comparable with each other. Without the cut, the modern era would be left with less than half the width instead of eighty-four per cent. **Below the axis there are two bands and not one**, and that is the point of the figure: the *AI winters* (dark grey) and the *dormancies of neural networks* (light grey) are different crises, with different causes, which overlap only in the middle. The first AI winter begins five years after the dormancy of the networks and ends six years before it: conflating them, as nearly all popular accounts do, destroys the sense of both. In each label the year in bold is the line closest to the timeline and the names of those involved sit just beyond; the thin leader connects the label to its point, and breaks where it passes behind another label.

It sounds obvious, and yet it is the kernel of the entire debate about artificial intelligence. If a machine only carries out instructions written by us, can it surprise us? And if it does surprise us, is that because it is intelligent or because we had not properly understood what we had ordered it to do?

Keep it to one side: a hundred and seven years later, somebody answers it.

19.2 1950: Turing asks the right question

Alan Turing publishes in the journal *Mind* a paper entitled *Computing Machinery and Intelligence*. He starts with the question “can machines think?” and then makes a move which is his real contribution: **he declares the question badly posed and replaces it**. Since nobody can define “thinking” without an argument, he proposes to replace it with a concrete test, the *imitation game*: a judge exchanges written messages with a human and with a machine, without seeing either, and has to guess which is which. If the judge can do no better than chance, the original question has lost its interest.

In the paper Turing lists and dismantles the predictable objections one by one, and devotes one of them explicitly to **Lady Lovelace**. His reply, boiled down, is this: saying that a machine only does what we order it to do is true only if we can foresee the consequences of our orders, and for non-trivial programs we cannot foresee them at all. Machines, says Turing, surprise him constantly.

The detail almost nobody tells

Two years earlier, in 1948, Turing had written for the National Physical Laboratory a report entitled *Intelligent Machinery* which stayed in a drawer until 1968. In it he describes what he calls *unorganised machines*: networks of elements connected **at random**, whose connections are then modified by a process he explicitly compares to the education of a child.
They are trainable neural networks, ten years ahead of Rosenblatt and almost forty ahead of back-propagation. Turing even proposes looking for good connections by an evolutionary method. The report was judged by his director to be little more than a schoolboy essay and was not published.

In that same 1950 paper Turing makes a prediction: by the year 2000, a machine with about a billion bits of storage would fool an average judge seventy per cent of the time in five minutes of conversation. On the storage he was right well ahead of time. On the rest, you be the judge.

19.3 1956: the conference that gives it a name

The phrase **artificial intelligence** was not born in a laboratory: it was born in a **funding application**.

On 31 August 1955 four researchers sign a proposal to the Rockefeller Foundation for a summer workshop at Dartmouth College, in New Hampshire. They are:

name	where they were	what they are known for
John McCarthy	Dartmouth College	will invent the LISP language
Marvin Minsky	Harvard	you will meet him again in 1969, against the perceptron
Nathaniel Rochester	IBM	designer of the IBM 701
Claude Shannon	Bell Labs	father of information theory

It is McCarthy who coins the phrase for the title of the proposal. He did so partly, as he later recounted, to mark a distance: the fashionable term was *cybernetics*, and using it would have meant working in Norbert Wiener's shadow. A new name served to claim a new field.

The workshop takes place in the summer of 1956 and lasts nearly two months. Passing through, some for weeks and some for a few days, are Allen Newell and Herbert Simon, who bring the *Logic Theorist*, a program able to prove theorems of logic on its own; Arthur Samuel, working on a draughts program that learns by playing; Ray Solomonoff and Oliver Selfridge.

The proposal also contains a sentence that has become the emblem of the optimism of those years. It says, in essence, that significant progress can be made on problems such as the use of language, the formation of abstract concepts and the self-improvement of machines, *if a carefully selected group of scientists work on it together for a summer*.

Seventy years later, those problems are still open.

Shannon and the mouse

Claude Shannon, who in 1948 had single-handedly founded information theory, in 1950 builds *Theseus*: a mechanical mouse that explores a maze, remembers the route and takes it straight the second time. The memory was telephone relays under the floor of the maze. It is one of the first learning machines of which a filmed demonstration exists, and it dates from 1950: eight years before the perceptron.

19.4 What deep learning really means

Deep learning means using networks with many layers. But it is not true, as you often hear, that adding layers was enough: adding layers was precisely the thing that did not work. Three categories of discovery were needed.

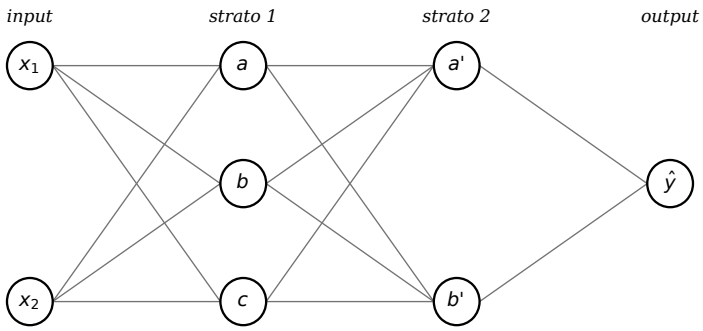


Figure 20: A network with two hidden layers. Every circle is a perceptron identical to the one you worked out by hand in the first few pages: multiply, add, decide. The networks of today are this picture with tens or hundreds of columns, and thousands of circles per column.

First category: the tricks that make deep networks learn

They are the ones in the table of the previous section, and others. Without the ReLU, without correct initialisation and without residual connections, a fifty-layer network simply does not learn. This is the technical contribution that made everything else possible.

Second category: architectures designed for the kind of data

A network in which every neuron is connected to all those in the previous layer is called *fully connected*, and it is the one you computed by hand. On an image of a million pixels it would have billions of weights in the first layer alone: unmanageable.

Yann LeCun, at the end of the nineteen-eighties, proposes **convolutional networks**: instead of a different weight for every pixel, the same little group of weights (a "filter") slides across the whole image. With a few hundred weights you can recognise an edge wherever it happens to be. By 1989 this idea is already reading postcodes on envelopes for the American postal service.

Yoshua Bengio, in 2003, makes the same move on language with *A Neural Probabilistic Language Model*. The idea is that each word should be represented not by a symbol but by a list of numbers (today we call it an *embedding*), so that similar words end up close together. It is the invention of neural language models, that is, the direct grandparents of today's LLMs.

Third category: the data and the machines

In 2012 **Alex Krizhevsky**, **Ilya Sutskever** and **Geoffrey Hinton** present AlexNet, a convolutional network trained on ImageNet (one million two hundred thousand labelled photographs) using two gaming graphics cards. It wins the annual image recognition contest by a margin nobody expected. That is the moment the world changes its mind.

In 2018 Bengio, Hinton and LeCun jointly receive the Turing Award, the equivalent of the Nobel Prize for computer science. In 2024 Hinton and Hopfield won the Nobel Prize in Physics **for foundational discoveries on machine learning with artificial neural networks**, while in the same year the developers of **AlphaFold** received the Nobel Prize in Chemistry for solving a fifty-year-old challenge in biology, **using AI to predict the three-dimensional structure of nearly every known protein** and so accelerate medical research.

19.5 How many winters there have been, and whose

Here one has to be precise, because this is the point at which nearly all accounts get confused, and the figure at the start of this section exists precisely to avoid that.

There are **two** AI winters, that is, two periods in which funding for the whole field dried up. And there are, separately, **two** dormancies of neural networks, which is a different thing: during the second of these, the field of artificial intelligence was in excellent health, it was the networks that were out of fashion.

period	what	what set it off
1974 – 1980	**first AI winter**	the Lighthill report of 1973, commissioned by the British government, concludes that AI has delivered nothing and that everything runs aground on combinatorial explosion. The United Kingdom dismantles the research almost everywhere. In the United States DARPA cuts the speech understanding programme in 1974
1987 – 1993	**second AI winter**	the market for LISP machines collapses, because ordinary workstations cost less and run just as well; expert systems turn out to be brittle and very expensive to maintain; DARPA cuts the Strategic Computing Initiative; the Japanese Fifth Generation project ends in 1992 without results
1969 – 1986	**neural networks dormant**	*Perceptrons* by Minsky and Papert, and above all the fact that nobody knew how to train more than one layer. It ends with backpropagation
1995 – 2006	**neural networks dormant again**	support vector machines work better, have a more solid theory and have no local minima. Networks stay out of the good conferences. It ends when people work out how to train deep networks

Note the dates: the first AI winter begins *five years after* the start of the dormancy of the networks and ends *six years before* it. They are two different crises, with different causes and different protagonists, overlapping in the middle. Calling the period 1969–1986 "the first AI winter" is convenient but wrong.

The second dormancy of the networks, the one from 1995 to 2006, these notes have already told in full: it is the section on why networks beat support vector machines. It simply was not called by its name.

Who invented the phrase "AI winter"

It is born in 1984, in a public debate at the annual meeting of the American association for artificial intelligence. Among those warning their colleagues is **Marvin Minsky**: the very man who in 1969 had set off the dormancy of neural networks warns that enthusiasm for expert systems is out of control again and that a chain reaction of pessimism, cuts and disappointment will follow.
He was right: three years later the market collapsed.
There is a coda to this story. When in 2006 deep networks started working again, the phrase *neural network* was so discredited in funding applications that people began using another one, which carried no trace of that past. That other phrase is **deep learning**, and it is the title of this section.

An open question, for class discussion

Two winters in seventy years averages one every thirty-five years, and a little over thirty have passed since the last one. Some argue that a third is on its way and some argue the opposite, with economic arguments that change from month to month.

I shall not give you my answer, because on this nobody has sufficient data. What I give you instead is the question that in my view discriminates, and it is the same one that applied in 1973 and in 1987: *are the promises being made today testable, and does whoever is making them have an interest in their being tested?*

20. Why the networks won, and not other methods

In the nineteen-nineties and the early two-thousands neural networks were by no means the best method. The champions were **support vector machines** (SVMs), invented by Vladimir Vapnik and colleagues. They were more elegant, theoretically more solid and in practice they almost always won. Then they lost. It is worth understanding why, because the answer explains the whole era we live in.

What an SVM does

An SVM looks for the *best possible* separating line, that is, the one with the widest margin from the nearest points. And when the data cannot be separated by a line, it uses a very refined trick called a *kernel*: it computes the similarities between all pairs of examples and pretends to work in a vastly larger space, where one line is enough.

The idea is splendid. The trouble is in "all pairs".

First reason: the cost grows with the square of the data

If you have n examples, the pairs are about $n^2/2$. With a thousand examples that is half a million calculations: nothing. With ten million examples it is fifty *thousand billion*: impossible, and there would not even be memory enough to write the table.

A neural network, by contrast, is trained in **small groups** (mini-batches): it takes 32 examples, does one gradient step, throws them away, takes another 32. The cost of one step does not depend on how much data you have in total, so the total cost grows *in proportion* to the number of examples, not to its square. You can train on data that would never fit in the computer's memory.

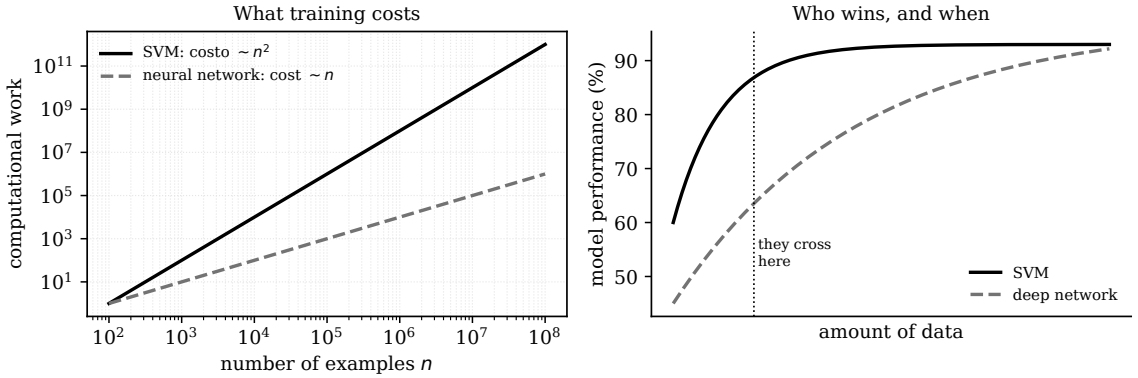


Figure 21: On the left: how the computational work grows (both axes have ticks that multiply by ten). On the right: who does better. With little data the SVM wins, and it is still an excellent choice today when the examples number a few thousand. With a lot of data the network keeps improving and overtakes.

Second reason: networks look as if they had been made for the graphics cards used in video games, the GPUs

This is the decisive reason. Look again at what a layer of a network does: every neuron computes $w_1x_1 + w_2x_2 + \dots$, and all the neurons do it *simultaneously*, on the same inputs, without waiting for one another. Lining up the weights of all the neurons means that an entire layer is a **matrix multiplication**, and nothing else.

A graphics card was born to work out where millions of triangles end up on the screen of a video game, which is again a matrix multiplication. It has thousands of little processors all doing the same operation on different data. The result is that a GPU trains a network a hundred times faster than an ordinary processor, and the leap came free, paid for by the video game industry.

The optimisation of an SVM, on the other hand, is a largely sequential process, made of choices that depend on the previous ones: it parallelises far worse.

Third reason: who chooses the features

With an SVM you have to decide the kernel yourself, that is, how to measure the similarity between two examples. On simple data this is an advantage, because you put in what you know. On a photograph it is a curse: nobody can write down by hand the formula that says when two images contain the same cat.

A deep network **learns the representation on its own**, and that is exactly what you saw happen in the figure of the hidden space for XOR. With a lot of data, hand-crafting always loses to learning from the data.

	SVM	**deep network**
cost with n examples	about n^2	about n
parallelisable	poorly	enormously, it is all matrices
representation	you choose it (the kernel)	the network learns it
with little data	often better	risks memorising
with a great deal of data	never gets there at all	keeps improving

21. From language models to attention

One last problem remained, and it was language.

To translate a sentence you need *recurrent* networks: they read one word at a time and carry along a summary of what they have read. These so-called RNNs (Recurrent Neural Networks) have two very serious problems. The first you already know: reading word by word, the gradient has to travel back through the whole sentence and vanishes, so the network forgets the beginning (this is exactly the problem studied by Hochreiter and by Bengio). The second is that reading in order is **sequential**, so it throws away the advantage of graphics cards: you cannot process word number fifty before you have finished number forty-nine.

2014: attention

Dzmitry Bahdanau, **Kyunghyun Cho** and **Yoshua Bengio** propose a beautiful solution. Instead of compressing the whole sentence into a single summary, they let the network, for every word it produces, **look back at all the source words and decide which ones to give weight to**. That mechanism is called **attention**.

And here is the point that concerns us: attention is still a weighted sum.

$$\text{result} \;=\; \alpha_1 \cdot (\text{word}_1) + \alpha_2 \cdot (\text{word}_2) + \dots$$

There is one single difference from the perceptron, but it is enormous: in the perceptron the weights w_i are fixed and learnt once and for all, whereas in attention the weights α_i **are recomputed for every sentence**, according to how much each word resembles the one being looked for. They are weights that change on the fly.

2017: attention is all you need

In 2017 eight researchers led by **Ashish Vaswani** publish a paper with a title that has become legendary: *Attention Is All You Need*.

The idea is radical: if attention works so well, let us throw away sequential reading and keep only attention. The network that comes out is called a **transformer** and it looks at all the words together, in parallel.

The advantage is not only in quality, it is above all in speed: without a sequence you can use graphics cards to the full, so you can train enormous models on all the text on the internet. From there onwards the road to today's language models (GPT, Claude, Gemini, DeepSeek and the rest) has mostly been a question of scale.

22. What survives of the perceptron inside an LLM

Let us close the circle. When you write a question to a language model, what actually happens inside? Much of what you have computed by hand.

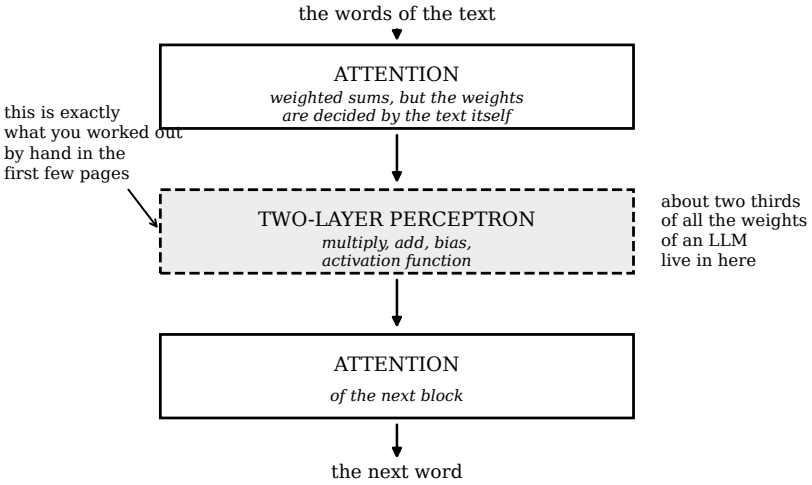


Figure 22: One transformer block, simplified. A real model stacks tens or hundreds of them. The dashed box is, literally, a two-layer perceptron.

in the perceptron	**in the transformer**	**what has changed**
$z = \sum w_i x_i + b$	the same, inside every attention head and in every layer	nothing: it is still multiply and add
fixed weights w_i	in attention the weights are recomputed for every sentence	the weights become dynamic
step function	ReLU, GELU and relatives	needed so that there is a slope to follow
the sigmoid gives a number between 0 and 1	the *softmax* gives a probability for each of the possible words	it is the sigmoid generalised to many choices
Rosenblatt's rule	gradient descent with backpropagation	it corrects always, not only when it is wrong
3 numbers to learn	hundreds of billions	only the scale

The piece that surprises people most is the one in the dashed box of the figure. In every block of a transformer, immediately after the attention, there is a module called the *feed-forward*: it takes the numbers coming out, multiplies them by a matrix of weights, adds a bias, applies an activation function, and multiplies by another matrix. It is exactly, without any conceptual difference, the two-layer perceptron you computed by hand a few pages ago. And in a modern language model **about two thirds of all the weights live in there**.

The calculation worth doing

In row R2 of epoch 1 you did three multiplications and three additions. A large language model, during its training, performs the same elementary operation roughly 10^{24} times. If you did it by hand, one calculation a second, never sleeping, it would take you about thirty thousand billion billion years. The operation has not changed: what has changed is only how fast we know how to repeat it.

And what was not in the perceptron

For honesty's sake, three things really are new, and none of them is obvious:

1. the **weights computed on the fly** in attention, which have no counterpart in Rosenblatt;
2. the idea of **learning the representation** instead of choosing it, which is the real legacy of LeCun and Bengio;
3. the fact that training on a huge amount of text to guess the next word produces, as a side effect, the ability to do things nobody trained the model for. Nobody predicted that, and to a large extent we still cannot explain it.

23. How an LLM works, piece by piece

In the previous section we said *what* survives of the perceptron inside a language model. Now we shall really take it apart, piece by piece, and do the calculations by hand on a real sentence. By the end of this section you will have carried out, with pencil and paper, every operation that a model like the ones you use every day performs billions of times a second.

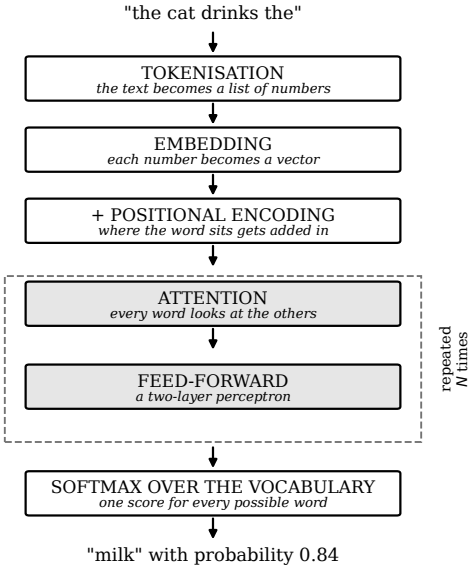


Figure 23: The complete stack. Every box is a subsection of this chapter. The dashed box is repeated tens or hundreds of times in a real model.

23.1 The task: guessing the next word

A language model does one thing only, and a more stupid one than it seems:

given a sequence of words, say which word comes next.

Nothing else. When you ask it a question, the model does not "answer": it continues the text, one word at a time, and every word it produces is stuck back on the end and used to produce the next one. All the apparent intelligence comes out of this game repeated millions of times.

Our example will be this one:

"the cat drinks the ..." $\longrightarrow$?

23.2 First piece: from text to tokens

The computer does not understand letters, it understands numbers. So we need a dictionary that associates a number to every piece of text. But which pieces?

- **One letter at a time?** The vocabulary would be tiny (some sixty symbols), but the sequences would become extremely long and the model would have to learn from scratch that "c-a-t" means something.
- **A whole word?** The sequences would be short, but the vocabulary enormous and, above all, *open*: as soon as a never-seen word turns up, say a surname or "cattishly", the model would not know what to do with it.

The solution is a middle way: the text is broken into **tokens**, pieces of words chosen so that frequent ones stay whole and rare ones get split. The most used method is called **BPE** (*Byte Pair Encoding*) and it works so simply that you can run it by hand.

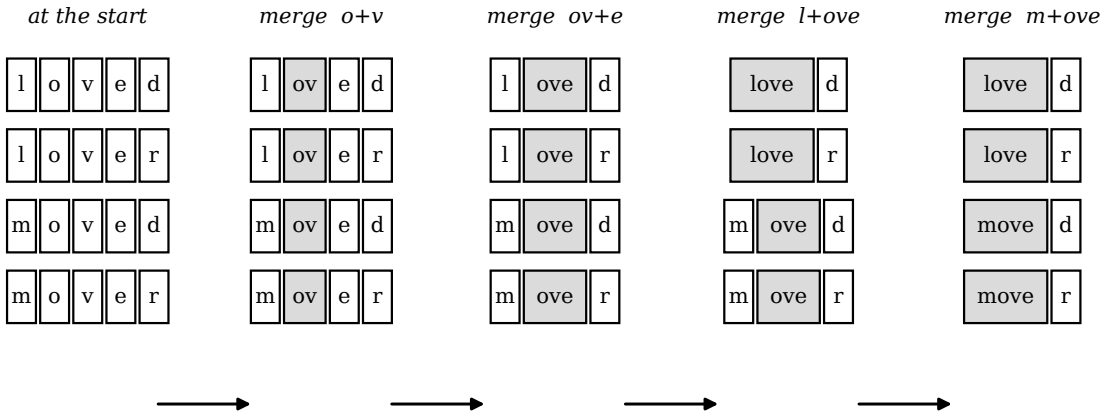
The BPE algorithm in three lines

1. Start by breaking everything into single letters.
2. Count all the pairs of neighbouring symbols and find the most frequent pair.
3. Merge that pair into a new symbol. Go back to point 2 and repeat until the vocabulary has the size you want.

BY HAND Let us do it on a mini-text of four words: *loved, lover, moved, mover*. Count the pairs of the first step yourself.

pair	how many times	
o + v	**4**	it appears in all four words
v + e	**4**	so does this one
l + o	2	
m + o	2	
e + d	2	
e + r	2	

Two pairs tie at 4, so we take the one that appears first: `ov`, which becomes a single symbol. We start again, and the following merges are `ov+e` = `ove`, then `l+ove` = `love` and finally `m+ove` = `move`.



the method knows nothing about grammar, yet it found the stem and the ending on its own

Figure 24: The first merges of BPE. The grey cells are the tokens born of the merges.

Look at the result: `love|d`, `love|r`, `move|d`, `move|r`. The algorithm has separated the stem from the ending. Nobody ever explained to it what a stem or an ending is: it merely counted frequent pairs, and the structure of the language emerged on its own. It is the same phenomenon you saw in the perceptron when it discovered by itself that distance is off-putting.

In our example the words are common, so each word is a single token:

token	the	cat	drinks	milk	dog	eats	bread
number	1	2	3	4	5	6	7

Our sentence therefore becomes the list of numbers $[1,\ 2,\ 3,\ 1]$. Note that the two "the" have the same number: for now the model has no way of telling them apart. We shall come back to that.

23.3 Special tokens, padding and masks

A model does not process one sentence at a time but a **group** of sentences together (it is called a *batch*), to make full use of the graphics card. But sentences have different lengths, and a graphics card wants rectangular tables.

The solution is **padding**: you take the length of the longest sentence in the group and fill up the others with a fake token, called `<PAD>`, which usually has the number 0.

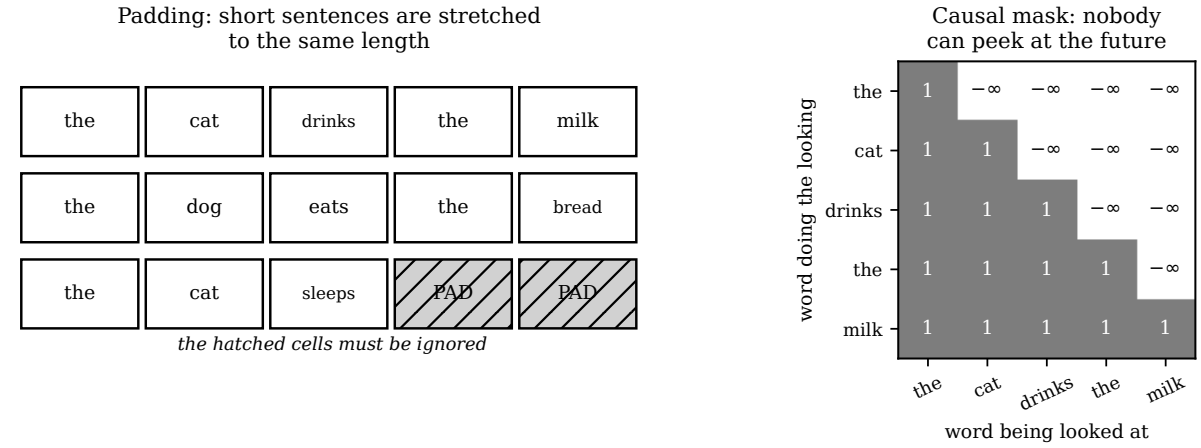


Figure 25: On the left the padding, on the right the causal mask.

But then the model has to be told to ignore those fake tokens, otherwise it would learn nonsense. A **mask** is used: a list of 1s and 0s as long as the sentence, where 0 means "there is nothing here". At the point where the model computes the scores (two pages from now) the scores of the masked positions are set to $-\infty$.

BY HAND Why $-\infty$ exactly? Because further on those scores end up inside an exponential, and

$$e^{-\infty} = 0$$

so that position receives weight exactly zero. It is not a dirty trick: it is the clean way of saying "this cell does not exist".

There is also a second mask, much more important, called the **causal mask**: while the model is learning to guess the next word, it must not be able to peek at the words that come afterwards, or the task would be trivial. So everything to the right is set to $-\infty$, giving the triangular shape in the figure. It is exactly the reason why an LLM writes from left to right and cannot go back and correct itself.

There are also other special tokens: `<BOS>` and `<EOS>` to mark the beginning and the end of a text, and `<UNK>` for the unknown.

23.4 Second piece: the embedding

We now have numbers, for instance cat $= 2$. Can we feed them to the network just as they are?

BY HAND No, and it is worth understanding why. If "cat" is worth 2 and "drinks" is worth 3, the network would compute things like $w \cdot 2$ and $w \cdot 3$, and would conclude that "drinks" is a sort of "cat" fifty per cent larger. But those numbers were only labels assigned arbitrarily: there is no ordering among the words of a dictionary.

The solution is to give each word not a number but a **vector**, that is, a list of numbers. This list is called an **embedding**, and each of its cells measures how much that word possesses a certain property.

In our example we shall use vectors of just four cells:

	article?	**animal?**	**action?**	**food?**
the	1	0	0	0
cat	0	1	0	0
dog	0	1	0	0
drinks	0	0	1	0
eats	0	0	1	0
milk	0	0	0	1
bread	0	0	0	1

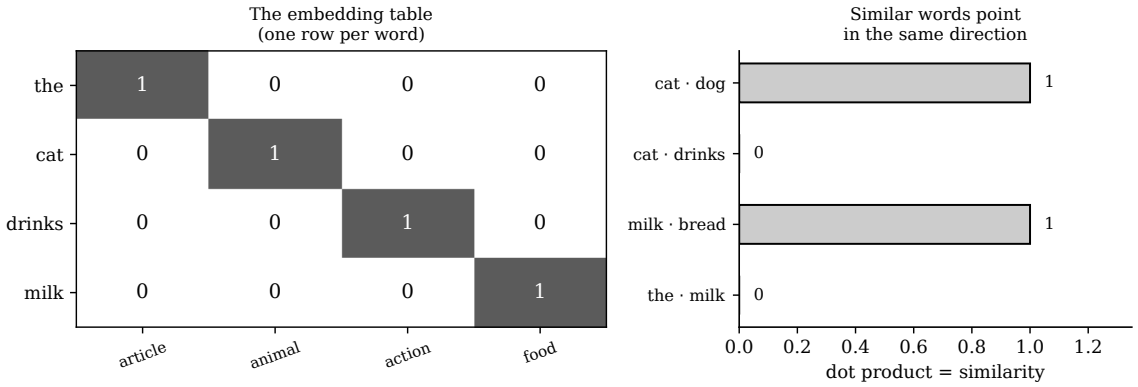


Figure 26: On the left the embedding table: it is a simple lookup table, one row per word. On the right the similarity between two words, measured with the dot product.

How you measure whether two words are similar

With the **dot product**, which is the same weighted sum as in the perceptron: you multiply the corresponding cells and add up.

BY HAND cat · dog $= 0 \cdot 0 + 1 \cdot 1 + 0 \cdot 0 + 0 \cdot 0 =$ **1**: very similar.

BY HAND cat · drinks $= 0 \cdot 0 + 1 \cdot 0 + 0 \cdot 1 + 0 \cdot 0 =$ **0**: nothing in common.

And now the important point, the one that explains why real models are so large. In our table **cat and dog have exactly the same vector**. With four cells there is no way of telling them apart: the model knows they are both animals and that is all. The same goes for milk and bread, and for drinks and eats.

To tell them apart you need more cells: one for “domestic”, one for “purrs”, one for “liquid”, and so on. A real model uses vectors of **thousands** of cells, and not one of them has a name chosen by a human: the network decides for itself which properties suit it, exactly as the perceptron decided for itself the sign of the distance weight.

Lookup tables are weights too

The embedding table is not written by hand by anyone: it is made of numbers that start out **random** and are learnt by gradient descent, like all the other weights. At the start of its training a model does not know that cat and dog resemble each other; it discovers it because they occur in the same sentences, and gradient descent brings their vectors together. This is the idea Yoshua Bengio introduced in 2003 and which made everything else possible.

23.5 Third piece: word order, with sine and cosine

There is a problem that looks small and is in fact large. The transformer looks at all the words *together*, in parallel: that is precisely what makes it fast. But then, as we have built it so far, it has no way of knowing what order they are in. For it,

"the cat drinks the milk" and *"the milk drinks the cat"*

are exactly the same thing: the same vectors, shuffled. And the two sentences do not mean the same thing at all.

So we need a way of writing "I am word number 3" inside the vector itself. The solution adopted by transformers is to add to every embedding a second vector that depends *only on the position*: the **positional encoding**.

Why writing the position number is not enough

The naive idea would be to add a cell containing $0, 1, 2, 3, \ldots$. It does not work, for three reasons:

1. on a long text that number becomes enormous and swamps all the other cells, which are worth about 1;
2. if you train on short sentences and then a long one arrives, the model meets numbers it has never seen and does not know what to do with them;
3. what the model needs is not so much the absolute position as the **distance** between two words, and that information is poorly recovered from a subtraction of large numbers.

The idea: two clocks turning at different speeds

We need numbers that always stay between -1 and $+1$ and that do not repeat too soon. Sine and cosine do exactly that.

Imagine a clock with two hands: a fast one, which turns a quarter of a turn per word, and a slow one, which turns an eighth. For each position you note where both are pointing. With two hands you have four numbers, that is, the sine and cosine of each.

BY HAND Fill in the table yourself. The fast hand turns $90°$ per word, the slow one $45°$. You only need values you already know: $\sin 0° = 0$, $\sin 45° = 0.71$, $\sin 90° = 1$, $\cos 0° = 1$, $\cos 45° = 0.71$, $\cos 90° = 0$.

position	**fast hand** ($90°$) sin	cos	**slow hand** ($45°$) sin	cos
0	0	$+1$	0	$+1$
1	$+1$	0	$+0.71$	$+0.71$
2	0	-1	$+1$	0
3	-1	0	$+0.71$	-0.71
4	0	$+1$	0	-1

Look at the row for position 4: the fast hand has come back exactly to where it was at position 0, but the slow one has not. That is what allows the two positions to be told apart. With ten hands of steadily decreasing speed you can distinguish tens of thousands of positions, and the numbers always stay between -1 and $+1$.

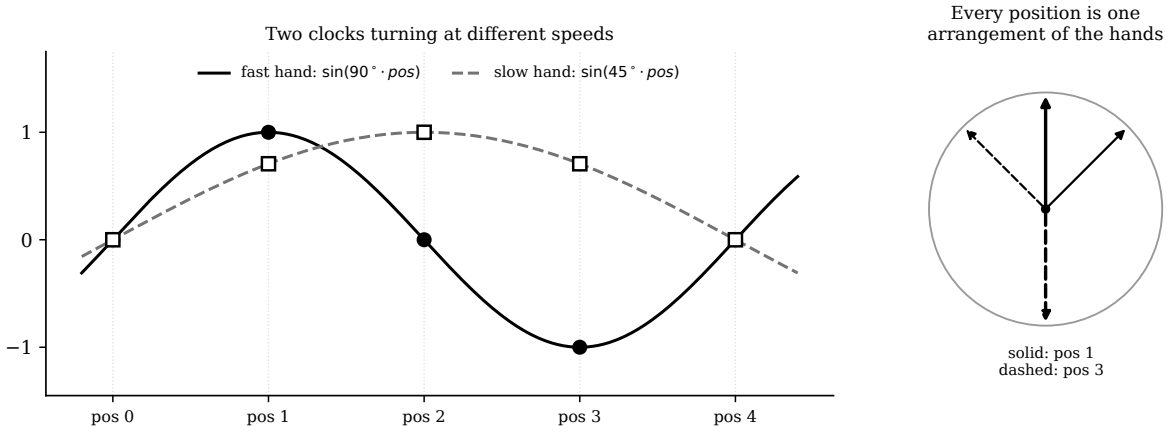


Figure 27: The two hands. On the left as waves, on the right as a clock. The fast hand on its own would come back to where it started every four words; it is the combination with the slow one that makes every position recognisable.

The property that makes all this clever

BY HAND Compute the dot product between position 1 and position 0:

$$[1,\ 0,\ 0.71,\ 0.71] \cdot [0,\ 1,\ 0,\ 1] = 0 + 0 + 0 + 0.71 = 0.71$$

Now between position 3 and position 2:

$$[-1,\ 0,\ 0.71,\ -0.71] \cdot [0,\ -1,\ 1,\ 0] = 0 + 0 + 0.71 + 0 = 0.71$$

The same result. And that is no accident: both pairs are one position apart. It can be shown in general that the dot product between two positional encodings depends *only on the difference* between the two positions, and not on where in the sentence they happen to be:

$$PE(p) \cdot PE(q) \ = \ \cos\big(90^\circ(p-q)\big) + \cos\big(45^\circ(p-q)\big)$$

(It is the cosine subtraction formula, $\cos\alpha\cos\beta + \sin\alpha\sin\beta = \cos(\alpha-\beta)$, applied to each pair of cells.)

This is the deep reason for choosing sine and cosine: the model, which always works with dot products, gets for free the information "these two words are three positions apart", which is exactly what grammar needs.

The real formula

In transformers the speed of hand number i is not chosen by eye but given by

$$PE(pos,\,2i) = \sin\Big(\frac{pos}{10000^{2i/d}}\Big), \qquad PE(pos,\,2i+1) = \cos\Big(\frac{pos}{10000^{2i/d}}\Big)$$

where d is the number of cells in the vector. With $d = 4$ the two divisors are $10000^0 = 1$ and $10000^{1/2} = 100$: the first hand turns a hundred times faster than the second. The speeds fall in geometric progression, like the digits of a number written in a strange base. Our version with 90° and 45° is exactly the same idea with angles that are easier to do in your head.

23.6 Fourth piece: attention, with the sums

We are at the heart of the transformer. The problem to solve is this: in the sentence "the cat drinks the ...", the last "the" on its own says nothing. To guess what comes next you have to *go and look at* the verb "drinks".

Attention does exactly this, and it does it with a mechanism that resembles a search in an archive. Every word produces three different vectors, obtained by multiplying its own vector by three learnt weight tables (W_Q, W_K, W_V):

name	**symbol**	**what it is for**
query	q	what I am looking for
key	k	what I offer to whoever is looking
value	v	what I hand over if I am chosen

The right image is that of a library. You arrive with a request (the *query*); every book has a label on its spine (the *key*); you compare the request with all the labels, and you take the contents (the *value*) of the books that match best. The difference from a real library is that you do not take one book only: you take a bit of all of them, in proportion to how well they match.

The calculation, step by step

In our example the three tables are tiny. The keys and queries will have just two cells, which we can read as "I am looking for an action" and "I am looking for a subject". The values will be the embeddings themselves.

The learnt tables turn out to be these:

	W_Q (who is looking)			W_K (who is offering)	
	seek action	seek subj.		am action	am subj.
article	1	0	article	0	0
animal	0	1	animal	0	1
action	0	0	action	2	0
food	0	0	food	0	0

BY HAND **Step 1: the query.** The one doing the looking is the last word, the "the" in position 3, whose embedding is $[1, 0, 0, 0]$. Multiplying by W_Q picks out the "article" row:

$$q = [1,\ 0] \qquad \text{that is: } \textit{I am looking for an action}$$

That makes sense: after an article you expect a noun, and to know *which* noun you have to look at the verb.

BY HAND **Step 2: the keys and the scores.** Every word of the sentence offers its key, and the score is the dot product with the query.

pos	**word**	**key** k	**the calculation** $q \cdot k$	**score**
0	the	$[0,\ 0]$	$1 \cdot 0 + 0 \cdot 0$	0
1	cat	$[0,\ 1]$	$1 \cdot 0 + 0 \cdot 1$	0
2	drinks	$[2,\ 0]$	$1 \cdot 2 + 0 \cdot 0$	**2**
3	the	$[0,\ 0]$	$1 \cdot 0 + 0 \cdot 0$	0

BY HAND **Step 3: the softmax.** The scores have to be turned into weights that sum to 1. We use the **softmax**, which is the big sister of the sigmoid: you raise e to each score and divide by the sum.

$$\text{weight}_i = \frac{e^{\,\text{score}_i}}{\sum_j e^{\,\text{score}_j}}$$

You only need $e^0 = 1$ and $e^2 = 7.39$:

$$\text{sum} = 1 + 1 + 7.39 + 1 = 10.39$$

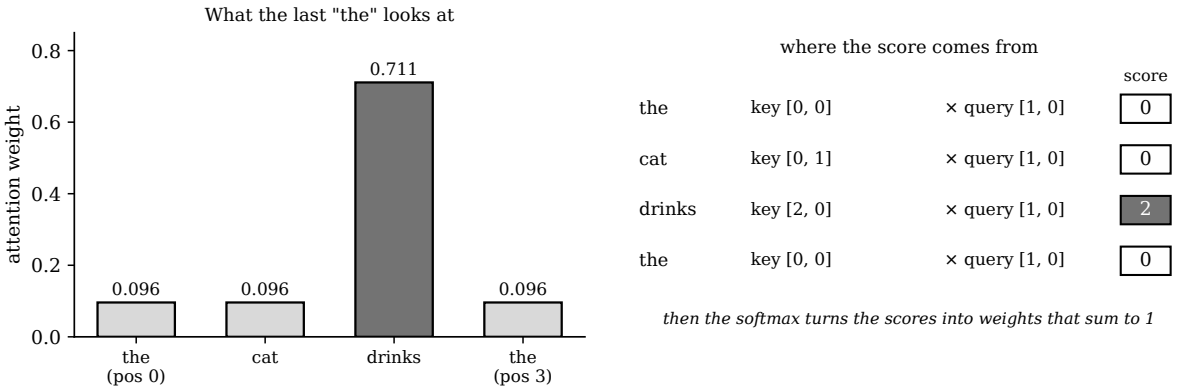


Figure 28: The attention weights of the last "the". 71% of its attention goes to the verb "drinks". Nobody told it to: it is the result of the learnt tables W_Q and W_K.

$$\text{weights} = \left[\frac{1}{10.39}, \frac{1}{10.39}, \frac{7.39}{10.39}, \frac{1}{10.39}\right] = [0.096,\ 0.096,\ \mathbf{0.711},\ 0.096]$$

Why the exponential in particular

Because it does three things at once, all of them needed. First, it makes every number positive, and a negative weight would make no sense. Second, it exaggerates the differences: a score of 2 against one of 0 becomes a ratio of seven to one, so the model chooses decisively instead of spreading itself thin. Third, it is differentiable everywhere, so gradient descent can be run on top of it. It is exactly the same reasoning that led to replacing the step function with the sigmoid.
In real models the scores are also divided by $\sqrt{d_k}$ (the square root of the number of cells in the key) before the softmax: without that division, with long vectors the scores would become enormous, the exponential would put all the weight on a single word and the gradient would die. It is the vanishing gradient problem again, following you even here.

BY HAND **Step 4: the weighted average.** Now we take the values (for us, the embeddings) and mix them with those weights.

$$\begin{aligned}\text{output} &= 0.096\,[1,0,0,0] + 0.096\,[0,1,0,0] + 0.711\,[0,0,1,0] + 0.096\,[1,0,0,0] \\ &= [0.096,\ 0,\ 0,\ 0] + [0,\ 0.096,\ 0,\ 0] + [0,\ 0,\ 0.711,\ 0] + [0.096,\ 0,\ 0,\ 0]\end{aligned}$$

$$\text{output} = [\,0.192,\ \ 0.096,\ \ \mathbf{0.711},\ \ 0\,]$$

Read it: the "action" cell is worth 0.711, all the others are small. The last "the", which on its own knew nothing, now carries with it the information *"there is a verb near me"*. This is all that attention does: it lets every word go and fetch the information it lacks, wherever in the sentence that information happens to be.

Why it is called multi-head

A real model does not do this calculation once, but ten or a hundred times in parallel with different W_Q, W_K, W_V tables. Each copy is called a **head** and specialises: one follows the verbs, one the agreement of gender and number, one the referents of pronouns. The results are then placed side by side. The heads do not wait for one another, so they all run together on the graphics card: it is another face of the parallelisation advantage discussed earlier.

23.7 Fifth piece: the feed-forward, that is, your perceptron

The vector that comes out of the attention now passes into a module you already know very well, because you computed it by hand twenty pages ago: a **two-layer perceptron**.

BY HAND In our example it has a single hidden neuron, which reads the action cell with a threshold of 0.5:

$$h = \text{ReLU}(0.711 - 0.5) = \text{ReLU}(0.211) = \mathbf{0.211}$$

and an output layer which, with a weight of 10, pours that value onto the food cell:

$$\text{output} = [\,0,\ 0,\ 0,\ 10 \cdot 0.211\,] = [\,0,\ 0,\ 0,\ \mathbf{2.11}\,]$$

The rule this perceptron has learnt, translated into English, is: *if there is a verb nearby, then expect a food*. The feed-forward module is the memory of the model: it is there that the regularities of the world are stored, while attention merely moves information about. And, as said in the previous section, in a real LLM about two thirds of all the weights sit right in here.

23.8 Sixth piece: from the stack to the prediction

Now we have to get from a vector of four cells to a probability for each of the seven words in the vocabulary. The most elegant way, and the one real models use, is called **weight tying**: you reuse the embedding table, computing the dot product between the output and the vector of every word. The reasoning is: *the right word is the one pointing in the same direction as what I am looking for*.

BY HAND The output is $[0, 0, 0, 2.11]$, that is, it points entirely in the "food" direction.

word	its vector	score	e^{score}	probability
the	$[1,0,0,0]$	0	1	0.046
cat	$[0,1,0,0]$	0	1	0.046
drinks	$[0,0,1,0]$	0	1	0.046
milk	$[0,0,0,1]$	$\mathbf{2.11}$	$\mathbf{8.27}$	$\mathbf{0.384}$
dog	$[0,1,0,0]$	0	1	0.046
eats	$[0,0,1,0]$	0	1	0.046
bread	$[0,0,0,1]$	$\mathbf{2.11}$	$\mathbf{8.27}$	$\mathbf{0.384}$
		sum	21.53	1.000

The model answers: *milk* with probability 38%, *bread* with probability 38%, everything else below 5%.

It is a remarkable result and at the same time a failure, and it is useful to understand both. Remarkable because the model has understood perfectly well that after "the cat drinks the..." comes a food: the five wrong words all have low probability. A failure because it cannot choose between milk and bread, and not out of stupidity: in our table *milk* and *bread* have exactly the same vector. The model cannot distinguish what it has no way of representing.

23.9 What training does

How do we measure how wrong the answer is? With the **cross-entropy loss**, which fortunately in our case reduces to a very simple formula:

$$L = -\ln P(\text{correct word})$$

BY HAND In our case $L = -\ln(0.384) = \mathbf{0.96}$.

Read it this way: if the model were completely certain and right, $P = 1$ and $L = -\ln 1 = 0$, zero loss. If it had given the correct word an almost zero probability, L would shoot off towards infinity. The loss punishes above all being confident and wrong, which is precisely the flaw you want to remove from a model.

Training on three sentences

The model learns on very many sentences. Let us take three:

	sentence	context	correct word
S1	the cat drinks the milk	the cat drinks the	milk
S2	the dog eats the bread	the dog eats the	bread
S3	the cat eats the bread	the cat eats the	bread

BY HAND With the four-cell embedding table, what is the loss on sentence S2? The reasoning is identical: "eats" has the same vector as "drinks", so the calculation is the same, and so is the answer: 38% to milk and 38% to bread. Loss 0.96 again.

Here is the point: **the loss cannot fall below** 0.96, however long you train, because the model does not have enough cells to distinguish the cases. We say the model is *at capacity*: it is not that it learns badly, it is that it has nowhere to put what it ought to learn.

What gradient descent does

Gradient descent acts on all the weights at once, embeddings included. And since the loss stays high as long as milk and bread are indistinguishable, the gradient pushes exactly where it is needed: it **widens the embedding table in a new direction**. Suppose the fifth cell becomes "liquid":

	article	animal	action	food	**liquid**
drinks	0	0	1	0	$+1$
eats	0	0	1	0	-1
milk	0	0	0	1	$+1$
bread	0	0	0	1	-1

BY HAND Redo the attention calculation: the weights are the same as before, so the output is now

$$[\,0.192,\ 0.096,\ 0.711,\ 0,\ \mathbf{0.711}\,]$$

because 71% of the weight sits on "drinks", which has $+1$ in the fifth cell. The feed-forward, on top of what it did before, lets the fifth cell through multiplied by 3, giving the output $[0, 0, 0, 2.11,\ 2.13]$. The scores become:

word	the calculation	score	probability
milk	$2.11 \cdot 1 + 2.13 \cdot (+1)$	$\mathbf{+4.25}$	$\mathbf{0.848}$
bread	$2.11 \cdot 1 + 2.13 \cdot (-1)$	-0.02	0.012
drinks		$+2.13$	0.103
the others		0 or less	below 0.013

Loss: $-\ln(0.848) = \mathbf{0.17}$, against 0.96 before.

This is, in miniature, the reason why models have become enormous. Not because anyone thinks bigger is cleverer, but because every distinction the model has to be able to make needs somewhere to live. The world contains a very great many distinctions to be made.

A note of honesty about this example

In the calculation above the model also gives 10% to "drinks", which after "drinks" makes no sense at all. It is not an arithmetic mistake: it is that our feed-forward has a single neuron and has no way of saying "do not repeat the verb". A real model has thousands of neurons per layer precisely so that it can learn prohibitions too, not just associations. I have left the flaw in rather than hiding it, because it shows well what you buy with size.

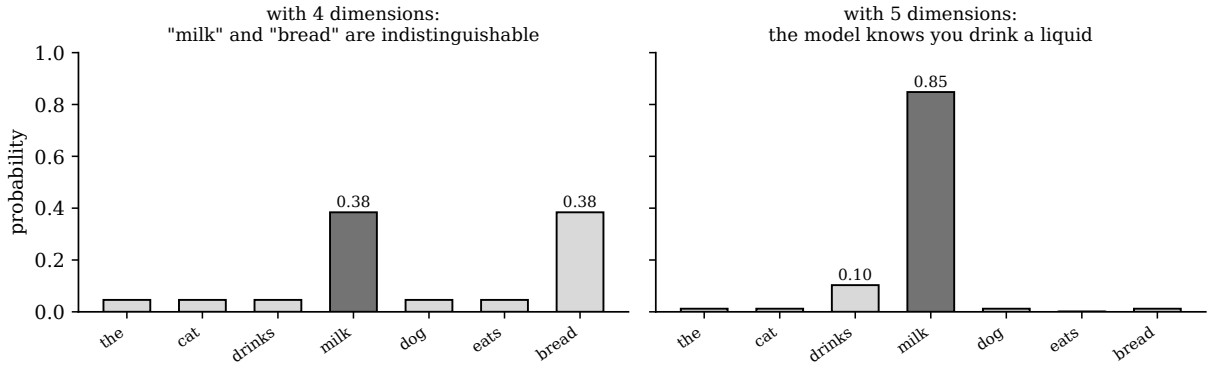


Figure 29: The same question, before and after. The only difference is one extra cell in the embedding table.

23.10 From our network to a real model

Here is everything that changes, and conceptually it is nothing.

	our example	a real LLM
tokens in the vocabulary	7	from 30 000 to 200 000
cells per vector	4	from 2 000 to 16 000
attention heads	1	from 16 to 128
stacked layers	1	from 40 to over 100
weights in total	about thirty	hundreds of billions
training text	3 sentences	thousands of billions of tokens
operations used	**multiply, add, threshold**	**the same**

And the complete cycle, when you write to a model, is this: your sentence is broken into tokens, every token becomes a vector, the position is added in, the whole thing passes a hundred times through the attention plus feed-forward pair, the softmax gives a probability to every word in the vocabulary, one is drawn, it is stuck back on the end of the sentence and **everything starts again from the beginning**. One word at a time, with no plan whatsoever about how the sentence will end.

The fact that something which looks like reasoning comes out of this procedure is, to this day, the thing researchers explain worst.

24. Exercises

1. With the final weights $w = (0.5,\ -0.2)$ and $b = -0.3$, work out z and $\hat{y}$ for a Saturday with 2 friends and 4 kilometres. Then with 5 friends and 10 kilometres.

2. Write the inequality $z \geq 0$ with those weights and solve it for x_2. How many kilometres at most are you willing to travel with 3 friends? And with 4?

3. Draw the line $x_2 = 2.5x_1 - 1.5$ on graph paper by finding two of its points. Then check that the point $(2,\ 5)$ lies above it and the point $(2,\ 3)$ below it, first by looking at the drawing and then by computing the sign of z.

4. Find values of w_1, w_2 and b that give a line with slope 2 and intercept 0, that is, exactly the true rule "every friend is worth two kilometres". Careful: there are infinitely many solutions. Why?

5. Redo epoch 1 starting from all-zero weights ($w = (0, 0)$, $b = 0$), still with $\eta = 0.1$. Do you reach the same final weights? Do you reach zero errors anyway?

6. What happens to the line if you multiply w_1, w_2 and b all three by 10? And do the perceptron's answers change? Explain why.

7. Prove, with the same reasoning used for XOR, that a single perceptron *can* implement the AND function. You only need to find three numbers that work and check the four rows.

8. In the backpropagation example, redo the update step using $\eta = 1$ instead of 2. Does the error still go down? By more or by less?

9. In backpropagation we found $\delta_a = -1/32$ starting from $\delta_{\text{out}} = -1/8$. If there were a third hidden layer, what would the next δ be worth, in the best case? And after five layers?

10. Why does the ReLU not suffer from the vanishing gradient? What happens instead to neurons whose output is always negative?

The eight things to remember

1. A perceptron always does three things and no more: multiply by the weights, add the bias, decide.
2. The bias is the threshold in disguise, and it is what lets the boundary avoid passing through the origin.
3. The boundary is a straight line because $w_1x_1 + w_2x_2 + b = 0$ is a first degree equation. The slope is $-w_1/w_2$, the intercept is $-b/w_2$, and the sign of w_2 says which side the yes is on.
4. The weights rotate the line, the bias shifts it. During training the line moves, and not always in the right direction at the first attempt.
5. A single layer draws a straight line, so there are problems (XOR) it will never be able to solve.
6. A hidden layer does not draw curves: it moves the points into another space, where a straight line is enough.
7. Backpropagation is the chain rule applied to a network: the "exchange rates" multiply along the backward path.
8. Deep learning is not "more layers": it is more layers *plus* the tricks that make them learn (ReLU, initialisation, residual connections), *plus* the right architectures, *plus* the data and the graphics cards.

For those who want to read the originals. Rosenblatt, *The Perceptron*, 1958. Minsky and Papert, *Perceptrons*, 1969. Rumelhart, Hinton and Williams, *Learning representations by back-propagating errors*, *Nature*, 1986. Bengio, Simard and Frasconi, *Learning long-term dependencies with gradient descent is difficult*, 1994. Bengio et al., *A Neural Probabilistic Language Model*, 2003. Krizhevsky, Sutskever and Hinton, *ImageNet Classification with Deep CNNs*, 2012. Bahdanau, Cho and Bengio, *Neural Machine Translation by Jointly Learning to Align and Translate*, 2014. Vaswani et al., *Attention Is All You Need*, 2017.

EXERCISE 11 · for those willing to take a risk

Rosenblatt invented the perceptron in order to imitate the human neuron. He could not have known it, but that intuition changed history.
Today the power of a network is more or less proportional to how many neurons you can fit inside it without making the graphics cards explode. All the progress of recent years has consisted in fitting in more of them.

Now it is your turn, and not with mathematics but with imagination.

Describe, at the level of the idea alone, a model that could surpass neural networks.

It does not have to imitate the human brain. It does not have to imitate any existing form of intelligence. It does not have to resemble anything you have read in these notes. It only has to be, as you imagine it:

parallelisable, simple, fast, efficient.

You do not need to know how to program it and you do not need to prove that it works. All you need to do is explain, in half a page, what elementary operation takes the place of the weighted sum, how it would learn from its mistakes, and why it ought to cost less.
Use your imagination. You may invent something that resembles nothing in existence. In this exercise, wrong ideas are worth as much as right ones: in 1958 nobody knew that multiplying and adding some numbers would lead this far.

25. An attempt at answering the last exercise

To be read only after you have tried. What follows is an attempt at answering exercise 11. It is not a serious research proposal: I have never trained it, I do not know how it performs, and further on you will find an honest list of the reasons why it might not work. Its purpose is to show you that one can reason about a new idea with the very tools you have used in these notes, and to give you a target to attack.

25.1 The question I started from

Looking back over all of these notes, one curious thing stands out. From 1958 to today almost everything has changed:

- the activation function: step, then sigmoid, then ReLU;
- the way of correcting: local rule, then gradient descent, then backpropagation;
- the architecture: one layer, then many, then convolutions, then attention.

But one thing has never changed in seventy years: **the elementary operation has always been the weighted sum**. Multiply every number by a weight and add. All the revolutions have concerned *how to adjust* those weights, never the fact that what was going on was multiplications and additions.

And that choice drags along with it every difficulty you have met. Gradients vanish because every layer *multiplies* the signal by a number smaller than one. Models are expensive because a dense layer requires n^2 multiplications. To train them you have to keep all the activations in memory, because sums are not invertible: from 7 you cannot get back to $3 + 4$.

So the question I asked myself is: *what if the elementary operation were not the sum, but the comparison?*

25.2 The building block: a pin

Take two numbers, a and b, and a single learnable parameter, a threshold θ. The rule is:

$$\text{if } a - b + \theta \geq 0 \quad \text{then } \textbf{swap}, \qquad \text{otherwise } \textbf{leave them alone}.$$

I call this contraption a **pin**, like the pins of a pinball machine.

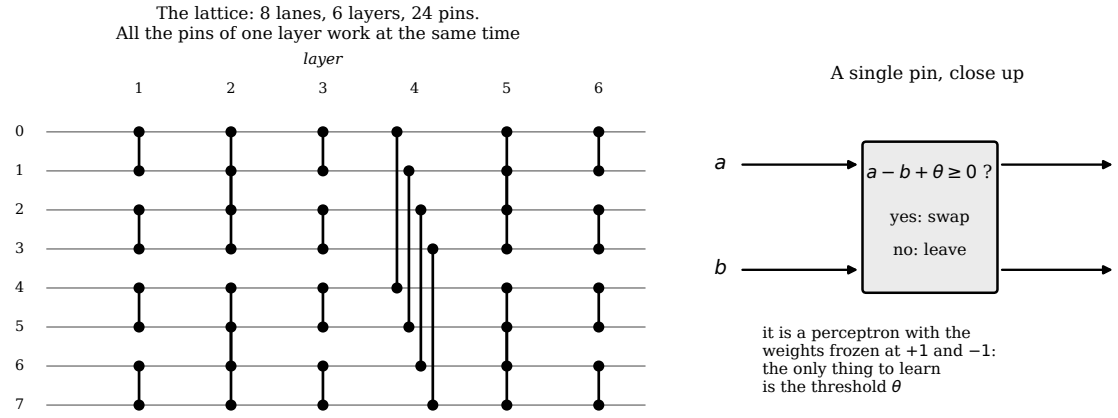


Figure 30: On the left the complete lattice on 8 lanes: 6 layers, 24 pins. The pins of one layer work on different pairs, so they all work at once. On the right the single pin.

BY HAND Look carefully at the condition $a - b + \theta \geq 0$ and compare it with the first page of these notes. It is

$$w_1 x_1 + w_2 x_2 + b \geq 0 \qquad \text{with} \qquad w_1 = +1, \quad w_2 = -1, \quad b = \theta$$

that is, **it is a perceptron with the weights frozen at $+1$ and -1, in which the only thing to learn is the bias**. I have not invented a new building block: I took Rosenblatt's and stripped almost everything off it. What is interesting is what happens when you put many of them together, and above all what you do with their result.

25.3 The lattice, and where the answer ends up

The pins are arranged in a lattice, like the pins of a pinball machine or the cells of a sieve: hence the name I have given it, **the Sieve**. The wiring diagram is not invented: it is that of Batcher's sorting networks, which have existed since 1968 and have depth $\log^2 n$ instead of n.

And here comes the part that is different. In a neural network the answer is the *value* that comes out of the last neuron. In the Sieve the values are never modified: a pin can only swap them around. So the answer cannot live in a value.

The answer lives in the arrangement.

A special value, a *marker*, is slipped in among the data. After the lattice you look at which slot it has ended up in: that slot number is the answer.

It is the idea of the abacus. The beads never change, only where they sit changes, and the position is the number.

BY HAND Let us try a tiny lattice, 4 lanes and 5 pins, with all the thresholds at zero. The pins are, in order: $(0,1)$ and $(2,3)$ in the first layer, $(0,2)$ and $(1,3)$ in the second, $(1,2)$ in the third. Input $[3,\ 7,\ 1,\ 9]$.

layer	pin	the calculation	result
1	$(0,1)$: $a=3$, $b=7$	$3-7+0=-4<0$	leave
1	$(2,3)$: $a=1$, $b=9$	$1-9+0=-8<0$	leave
		after layer 1:	$[3,\ 7,\ 1,\ 9]$
2	$(0,2)$: $a=3$, $b=1$	$3-1+0=+2\geq 0$	**swap**
2	$(1,3)$: $a=7$, $b=9$	$7-9+0=-2<0$	leave
		after layer 2:	$[1,\ 7,\ 3,\ 9]$
3	$(1,2)$: $a=7$, $b=3$	$7-3+0=+4\geq 0$	**swap**
		after layer 3:	$[1,\ 3,\ 7,\ 9]$

With all the thresholds at zero the lattice sorts. But the thresholds are not at zero: they are learnt, and every pin has its own. A θ different from zero means "swap only if a beats b by at least this much", that is, it introduces a personalised handicap at that precise point of the lattice. With all the thresholds different the lattice no longer sorts: it does something else, and that something else is the function it has learnt.

25.4 How it learns

And here the problem that runs through all of these notes comes back: if the answer is wrong, *whose fault is it*?

Rosenblatt	the blame lies with the switched-on inputs. That is why the correction is proportional to x_i.
backpropagation	the blame is divided by the chain rule, and to do that you have to touch *every* weight.
the Sieve	the blame lies with the pins the marker actually touched along its way. There are $\log^2 n$ of them, not all of them.

The marker, crossing the lattice, meets exactly one pin per layer. If it has ended up too far to the left, all the pins along its path had a threshold slightly too low; if too far to the right, slightly too high. The correction is embarrassingly simple and follows Rosenblatt's closely:

$$\theta \;\leftarrow\; \theta \pm 1 \qquad \text{for the pins that were touched only}$$

BY HAND Let us see it on a single pin. Task: answer 1 if a beats b by at least 3. Initial threshold $\theta = 0$.

a	b	y	θ	$z = a - b + \theta$	$\hat{y}$	**new** θ
5	1	1	0	$+4$	1	*correct*
3	2	0	0	$+1$	1	-1
7	2	1	-1	$+4$	1	*correct*
4	4	0	-1	-1	0	*correct*
6	1	1	-1	$+4$	1	*correct*
2	5	0	-1	-4	0	*correct*
5	3	0	-1	$+1$	1	-2
8	1	1	-2	$+5$	1	*correct*

End of the first epoch: 2 errors, $\theta = -2$. In the second epoch it gets only the row $(5, 3)$ wrong and θ falls to -3. In the third epoch it gets nothing wrong at all: with $\theta = -3$ the condition $a - b - 3 \geq 0$ means exactly $a \geq b + 3$, which was the rule to be learnt.

It is the very same table as epoch 1 in the perceptron section, with the same shape and the same happy ending. Except that here there is only one number to learn, and it is an integer.

25.5 The three things that make it interesting

First: the gradient cannot vanish

This is the property that convinces me most, because it settles outright the problem of the vanishing gradient section.

The gradient vanishes because every layer *multiplies* the signal by a factor smaller than one: with the sigmoid, at most 0.25, and 0.25^{20} is zero. In the Sieve there is no multiplication at all, so there is no factor. A value that enters the lattice comes out identical, only in a different slot. A hundred layers consume it no more than one does.

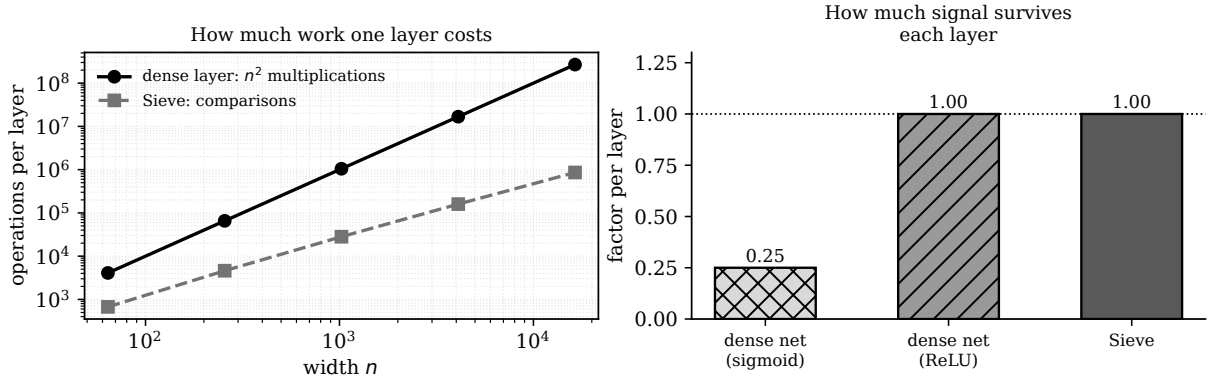


Figure 31: On the left the computational work for one layer. On the right how much signal survives each layer: in the Sieve it is exactly 1, because the values are never rescaled.

Second: it costs little, and in the right currency

A dense layer on n numbers requires n^2 multiplications. The lattice requires $\frac{n}{2}\log^2 n$ comparisons, and a comparison is the cheapest thing a processor knows how to do.

n	multiplications (dense)	comparisons (Sieve)	depth
1 024	1 048 576	28 160	55
16 384	268 435 456	860 160	105

At $n = 16\,384$ that is three hundred times fewer operations, and cheaper ones one by one at that. On top of that the parameters are small integers, not floating point numbers: the memory collapses.

Third: you can go backwards without having remembered anything

This seems to me the most elegant consequence, and I had not foreseen it when I began thinking about it.

To train an ordinary network you have to keep all the activations of the forward pass, because they are needed in the backward pass. It is the largest item of expenditure in training large models. The reason is that a sum cannot be inverted: knowing that the result is 7 does not tell you whether it came from $3 + 4$ or from $5 + 2$.

A swap, on the other hand, is inverted by doing it again. All you need to record, for every pin, is **one bit**: did I swap or not. With those bits you retrace the lattice backwards and reconstruct the input exactly.

BY HAND In the 4-lane lattice above the bits were $[0, 0, 1, 0, 1]$. Start from $[1, 3, 7, 9]$, redo the pins in reverse order swapping wherever the bit is 1, and you get $[3, 7, 1, 9]$ back. Try it.

Five bits instead of twelve floating point numbers, in this tiny example. On a real model the ratio would be one to hundreds.

25.6 Why it might not work

Now the part that in a research proposal is worth more than all the others.

1. **The Sieve cannot create anything new.** The numbers that come out are exactly those that went in, in a different order. It cannot take averages, it cannot add two pieces of information into a third. This is a serious limitation and not a detail: it means the Sieve is not a replacement for neural networks, but a different kind of model, good for different problems.

2. **It computes order functions, not linear functions.** It can answer questions of the kind "which is the biggest", "how many are above the threshold", "in what order are they". On a problem that is genuinely a weighted sum, like our cinema on the first page, it does badly.

3. **It has little capacity per layer.** The information an arrangement of n elements can hold is $\log_2(n!)$ bits: for $n = 1024$ that is about $8\,800$ bits, one kilobyte. A dense layer of the same width has a million weights. You would need a great many lattices in parallel to compensate, and at that point the advantage in cost shrinks.

4. **The local rule is an approximation, and might not be enough.** Saying that the blame lies only with the pins the marker touched is convenient but not entirely true: the pins it did *not* touch also decided which numbers the marker would meet. It is exactly the credit assignment problem that held networks up for seventeen years, turning up here in disguise. It may be that a local rule is sufficient in practice, as it was for Rosenblatt on separable problems; it may equally be that training seizes up on deep lattices. I do not know, and without trying nobody does.

5. **The parameters are integers, so the error landscape is a staircase.** There is no smooth descent to follow: you proceed in jumps, and jumps can go round in circles.

25.7 What already exists, for honesty's sake

No idea is born out of nothing, and it would be improper to let you believe this one fell from the sky. Here are the close relatives I know of:

sorting networks (Batcher, 1968)	the wiring of the lattice is taken straight from there, and the depth $\log^2 n$ is a result of theirs
Beneš permutation networks	routing signals with switches rather than with arithmetic is an old idea from telecommunications
Tsetlin machines, weightless networks	models that learn discrete rules instead of real numbers
differentiable sorting, max-pooling	pieces of modern networks that are already, in effect, order operations

What I am not aware of ever having been put together is the combination: a lattice of comparators with a learnt threshold, trained with a local rule along the path, in which the answer is read from the position of a marker rather than from a value. If one of you finds that it already exists, that is a more useful discovery than the idea itself: it means you can go and read how it turned out.

25.8 The bet

There is one argument in favour worth leaving you with, because it is the kind of reasoning that gives birth to ideas.

Look at what modern networks do at their decisive moments. The softmax mostly serves to make the largest value emerge. Attention assigns weights according to which is most similar, that is, it produces a ranking. The final prediction of an LLM is essentially a sorting of the vocabulary, of which one then looks at the top.

What if a good part of what a network laboriously computes
with billions of multiplications were, in the end, order information?

If that were true, computing it directly in the world of orderings instead of extracting it from arithmetic would be an enormous shortcut. If it were false, this idea is of no use at all.

I do not know which of the two. But it is a question that can be answered with an experiment, and questions of that kind are the only ones that move anything forward.

What to take away from this chapter

Not the idea of the Sieve, which almost certainly has some flaw I have not seen. Rather the method, which is always the same:

1. look for the assumption nobody questions because it has been there too long (here: that computing means multiplying and adding);
2. change that one, and only that one;
3. see what falls down and what stays standing;
4. list honestly the reasons why your idea ought to fail, before somebody else does it for you.

That is exactly what Rosenblatt did in 1958, and the loveliest part is that he did not need to know in advance that it would work.